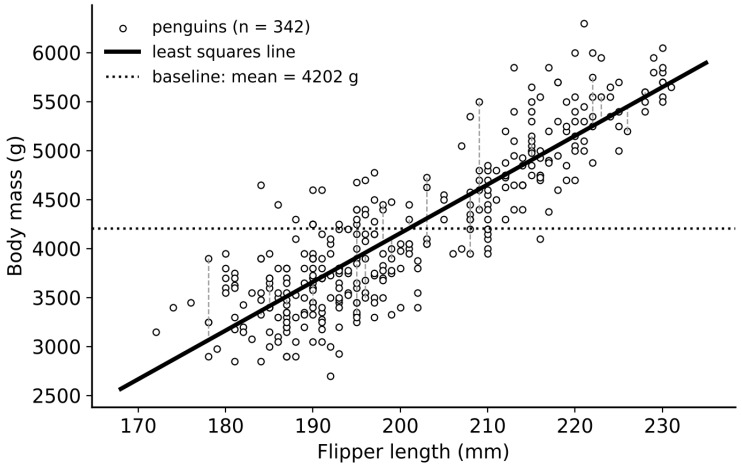
MACHINE LEARNING · WEEK 2 · LECTURE 2

From Risk Minimisation to Linear Regression

Maths revision · empirical risk minimisation · baselines · least squares · gradient descent · ridge and lasso

Behnam Kiani

Fall Semester 2026–2027



$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$$

Fit a line by minimising error — exactly, or step by step.

Where this lecture fits in the new syllabus

Syllabus week	Topic	Where it is covered
1	Learning paradigms	Lecture 1 ✓
1	Problem formulation	Lecture 1 ✓ (T–E–P) · formalised today
1	**Mathematical revision**	**Today — Part 01**
1	**Empirical risk minimisation**	**Today — Parts 02–03**
2	**Linear regression · least squares**	**Today — Part 04**
2	**Gradient descent**	**Today — Part 05**
2	**Ridge and lasso regularisation**	**Today — Part 06**



Today's lab

- NumPy exercises
- Baseline model
- Regression from scratch
- Gradient descent from scratch
- Ridge & lasso



Next week

Logistic regression · maximum likelihood · multiclass · Naive Bayes

From Lecture 1 to Lecture 2: making it precise

LECTURE 1 (words)		LECTURE 2 (mathematics)
A model is a function with knobs	→	hypothesis $h(x) = w^T x$ from a class $\mathcal{H}$
Learning = reducing error	→	minimise the empirical risk $\hat{R}_n(h)$
Adjust the knobs a little at a time	→	gradient descent $w \leftarrow w - \eta \nabla J(w)$
We care about new data	→	true risk $R(h)$ vs empirical risk $\hat{R}_n(h)$

Today's learning objectives

1. Use ML notation, linear algebra, gradients, convexity and expectation
2. State empirical risk minimisation and formulate a problem as $(\mathcal{X}, \mathcal{Y}, \mathcal{H}, \ell)$
3. Derive a baseline: why the mean minimises squared error
4. Derive the normal equations and interpret them geometrically
5. Implement gradient descent; choose η; scale features; use SGD
6. Explain ridge and lasso: shrinkage, sparsity, choosing λ

Agenda

01 

Maths toolkit

02 

Risk minimisation

03 

Baselines

04 

Linear regression & least squares

05 

Gradient descent

06 

Ridge & lasso

07 

Lab & next steps

01

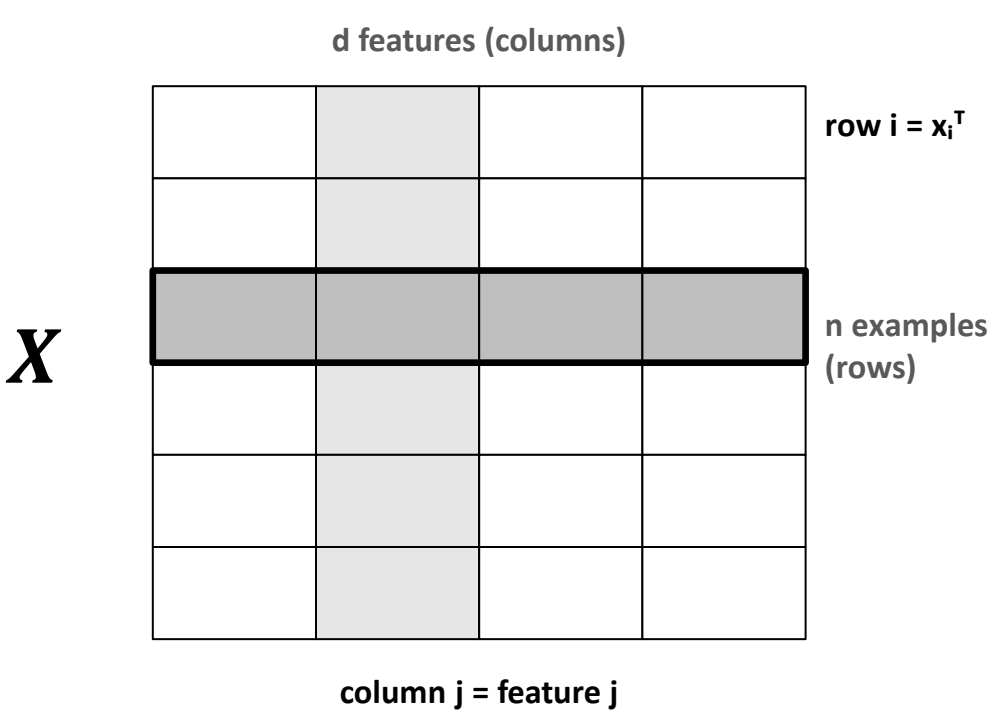
Maths toolkit

The linear algebra, calculus and probability you need



Notation: scalars, vectors, matrices

$$\mathbf{x} = (x_1, x_2, \ldots, x_d)^\top \in \mathbb{R}^d$$

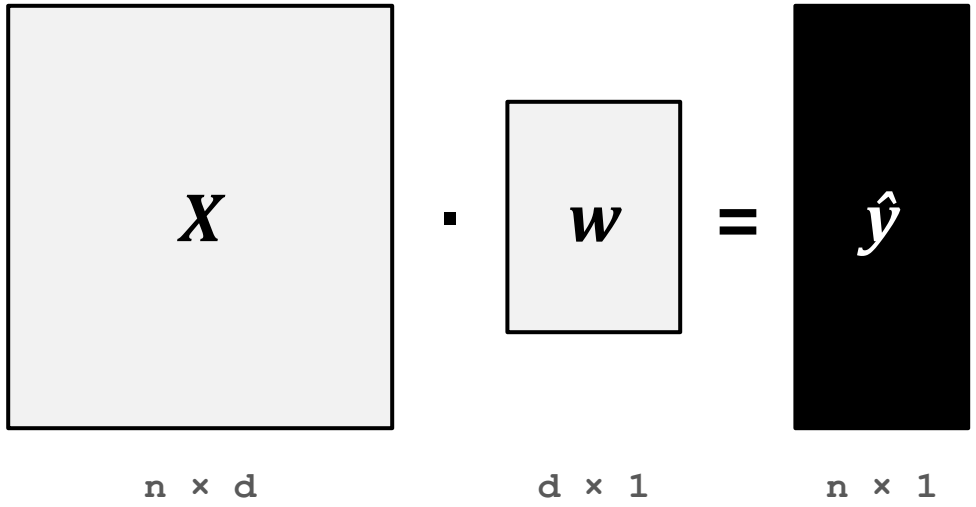


Symbol	Meaning	Penguins
n	number of examples	342
d	number of features	1 (flipper length)
$X \in \mathbb{R}^{nxd}$	data matrix	342 × 1
$y \in \mathbb{R}^n$	targets	body mass (g)
$w \in \mathbb{R}^d$	parameters (weights)	slope (+ intercept b)
$\hat{y}$	prediction	predicted mass
η, λ	learning rate, penalty	hyperparameters

Most from-scratch bugs are shape bugs: print .shape often.

Matrix–vector products: all predictions at once

$$\hat{\mathbf{y}} = X\mathbf{w}, \quad X \in \mathbb{R}^{n \times d},\ \mathbf{w} \in \mathbb{R}^{d},\ \hat{\mathbf{y}} \in \mathbb{R}^{n}$$



inner dimensions match → result n × 1

```
X.shape          # (342, 2)
w.shape          # (2,)
y_hat = X @ w    # matrix product
y_hat.shape      # (342,)

X * w            # element-wise! (342, 2)
```

Classic bug: * instead of @

X * w multiplies element by element and broadcasts to shape (n, d) — no error, wrong answer. Use X @ w.

Dot products and norms

$$\mathbf{w}^\top \mathbf{x} = \sum_{j=1}^{d} w_j x_j$$

$$\|\mathbf{w}\|_2^2 = \sum_j w_j^2 \qquad \|\mathbf{w}\|_1 = \sum_j |w_j|$$

```
a = np.array([3, -4])
np.linalg.norm(a)      # 5.0   (L2)
np.linalg.norm(a, 1)   # 7.0   (L1)
np.sum(a ** 2)         # 25    (squared L2)
w @ x                  # dot product
```



Linear score

$w^\top x$ is a weighted sum of the features — the heart of every linear model.



Orthogonality

$a^\top b = 0 \Leftrightarrow$ perpendicular. Key to the geometry of least squares (Part 04).



Two norms, two penalties

L2 → ridge regression · L1 → lasso regression (Part 06).

Transpose, inverse and solving linear systems

Object	Shape	Note
X	$n \times d$	data matrix
X^T	$d \times n$	transpose
X^TX	$d \times d$	small, symmetric
X^Ty	d	feature–target products
$(X^TX)^{-1}$	$d \times d$	exists only if full rank

When the inverse fails

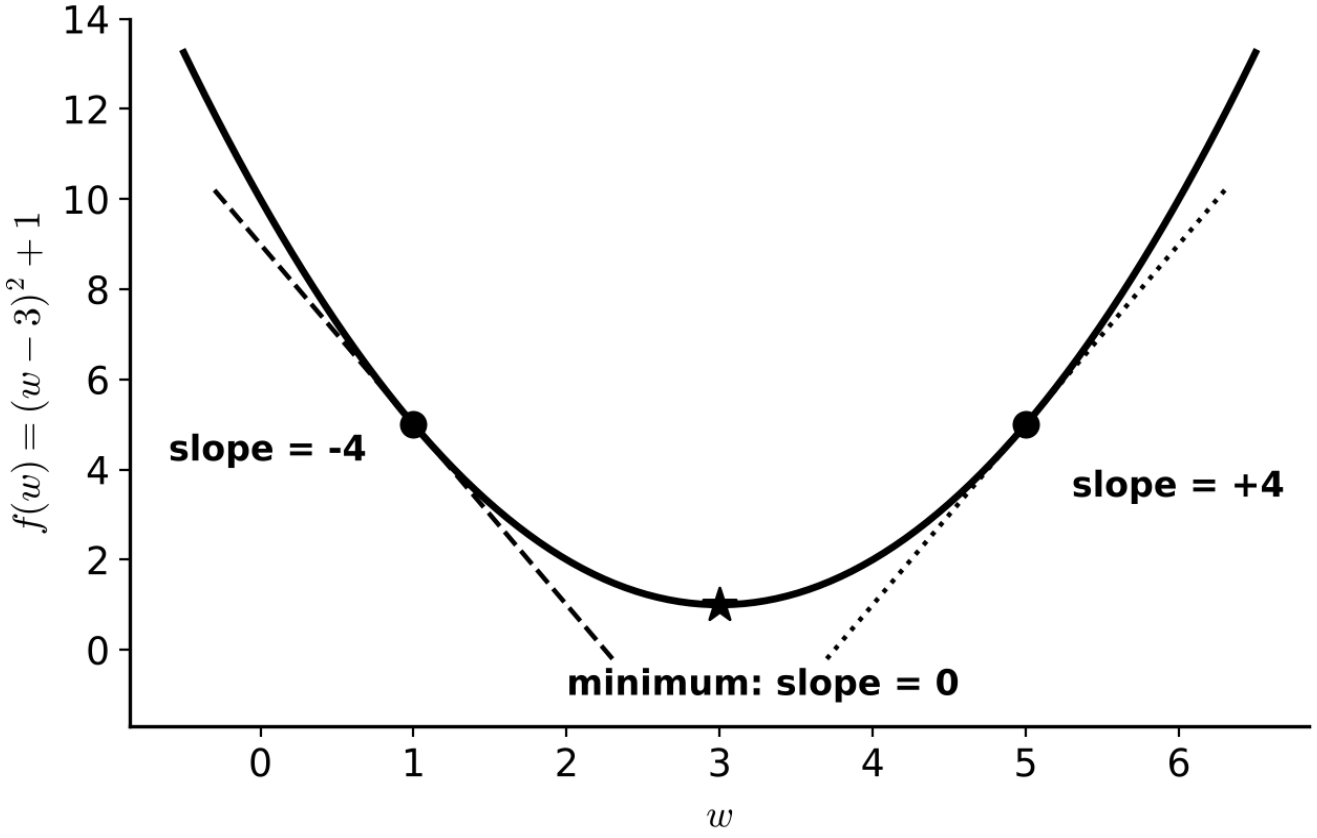
Collinear features (cm and inches) or $d > n \rightarrow X^TX$ singular.

```
A = X.T @ X                    # (d, d)
b = X.T @ y                    # (d,)
w = np.linalg.solve(A, b)      # ✓ preferred
w = np.linalg.inv(A) @ b       # ✗ slower, less
accurate
w, *_ = np.linalg.lstsq(X, y, rcond=None)
```

Rule of thumb

Solve, don't invert. And when X^TX is singular or nearly so, regularise (ridge, Part 06).

Derivatives: slopes tell us which way is down



$$f(w) = (w-3)^2 \quad \Rightarrow \quad f'(w) = 2(w-3)$$

Slope −4 at w = 1

decreasing → move right

Slope +4 at w = 5

increasing → move left

Slope 0 at w = 3

the minimum: f′(w) = 0

Gradients and the chain rule

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_d} \right)^\top$$

$$\frac{d}{dw} g(h(w)) = g'(h(w))\, h'(w)$$

Function	Gradient / derivative	At a point
$w_1^2 + 3w_2^2$	$(2w_1, 6w_2)$	(1, 1) → (2, 6)
$(w\,x - y)^2$	$2(w\,x - y) \cdot x$	w=2, x=3, y=5 → 6
$a^\top w$	a	constant

Direction of steepest ascent

The gradient points uphill. Gradient descent steps along −∇f, the steepest way down.

Chain rule → backprop

Neural networks are long chains of functions; backpropagation (Week 11) is the chain rule done systematically.

The matrix calculus you need today

1. **Expand:** $\|Xw - y\|^2 = w^TX^TXw - 2w^TX^Ty + y^Ty$
2. **Rule 2 on first term:** $\nabla(w^TX^TXw) = 2X^TXw$
3. **Rule 1 on second term:** $\nabla(-2w^TX^Ty) = -2X^Ty$
4. **Add (last term is constant):** $\nabla = 2X^TXw - 2X^Ty$

$$\nabla_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|_2^2 = 2X^{\mathsf{T}}(X\mathbf{w} - \mathbf{y})$$

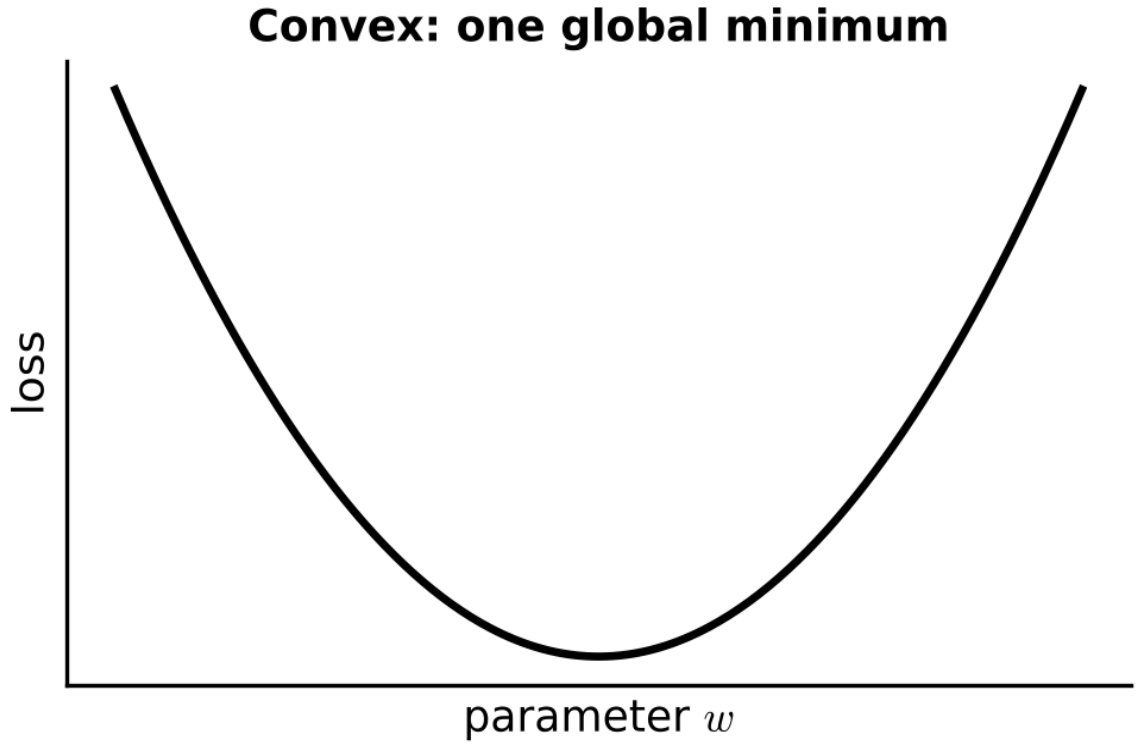
Rule 1

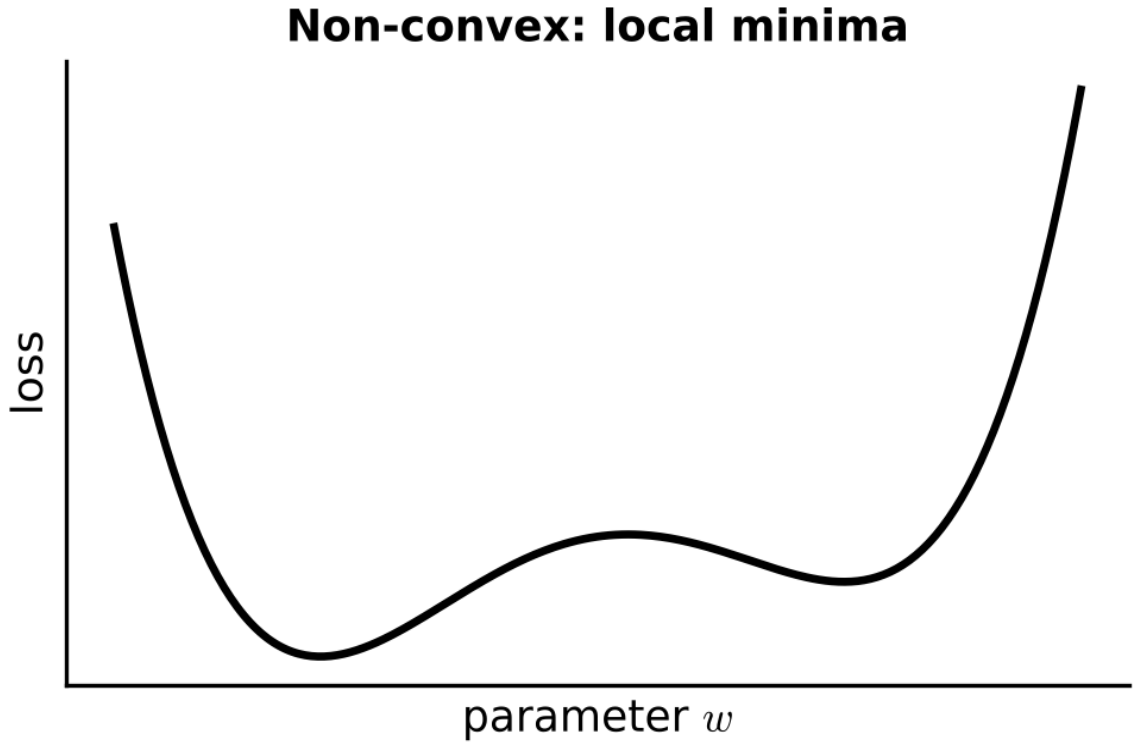
$\nabla(a^Tw) = a$
like $d(aw)/dw = a$

Rule 2

$\nabla(w^TAw) = 2Aw$
(A symmetric)
like $d(aw^2)/dw = 2aw$

Convexity: when “downhill” is guaranteed to work





Today's losses are convex: one bowl, one minimum

Neural networks: non-convex (Week 11)

Probability revision: expectation, variance and noise

$$\mathrm{E}[Y] = \sum_{y} y\, p(y) \qquad \mathrm{Var}[Y] = \mathrm{E}\big[(Y - \mathrm{E}[Y])^2\big]$$

$$y = \mathbf{w}^{\mathsf{T}}\mathbf{x} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \;\Rightarrow\; \text{max likelihood} \equiv \text{min squared error}$$

3.5

expected value of a fair die

σ^2

noise variance = irreducible error

$n \to \infty$

sample average → expectation

Least squares is not arbitrary: it is maximum likelihood under Gaussian noise — the bridge to Week 3.

Quick check: maths toolkit

1 **X is 342×2, w has length 2. Shape of X @ w?**

→ *(342,) — one prediction per penguin*

2 **Derivative of $5w^2$ at w = 2?**

→ *10w → 20*

3 **$\|(3, -4)\|_2$ and $\|(3, -4)\|_1$?**

→ *5 and 7*

4 **Is $|w|$ convex?**

→ *Yes — but not differentiable at 0*

5 **Expected value of a fair die?**

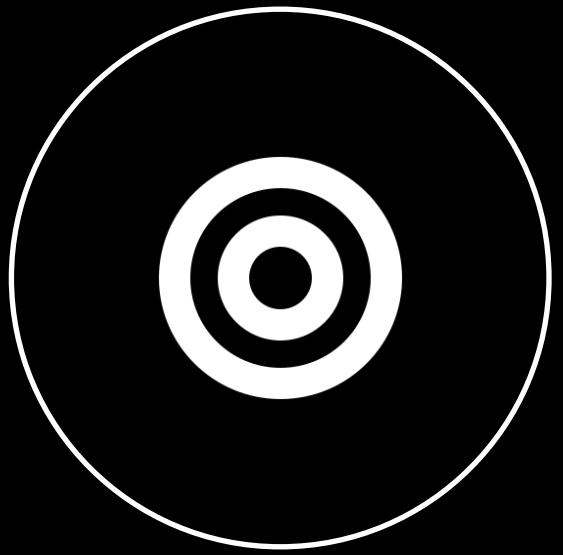
→ *3.5*

6 **X is 442×11. Shape of X^TX?**

→ *11 × 11*

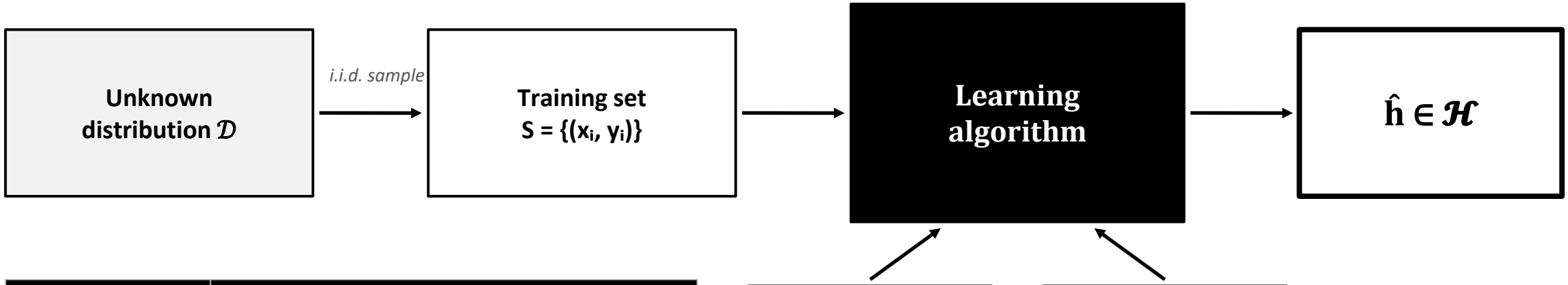
02

Learning as risk minimisation



One principle behind almost every learning algorithm

Supervised learning, formally



Ingredient	Penguins example
$\mathcal{X} \to \mathcal{Y}$	flipper length (mm) → body mass (g)
$\mathcal{H}$	all lines $h(x) = wx + b$
ℓ	squared error $(\hat{y} - y)^2$

Every method in this course = a choice of $\mathcal{H}$, ℓ and a search procedure.

Learning paradigms, formally

Paradigm	Given	Optimise	Examples	Weeks
Supervised	pairs (x_i, y_i)	average loss $\ell(h(x_i), y_i)$	linear & logistic regression, trees, SVMs	2–7
Unsupervised	inputs x_i only	structure objective	k-means, GMMs, PCA	9–10
Reinforcement	an environment	expected cumulative reward	game playing, control	15
Self-supervised	inputs; labels made from them	prediction of hidden parts	language models, embeddings	13–14

All four are optimisation problems — what changes is what is optimised.

Problem formulation: a checklist

Question	Penguins answer
1 Input $\mathcal{X}$?	flipper length (mm), one feature
2 Output $\mathcal{Y}$?	body mass (g): real number → regression
3 Hypothesis class $\mathcal{H}$?	straight lines $\hat{y} = wx + b$
4 Training loss ℓ?	squared error $(\hat{y} - y)^2$
5 Evaluation metric?	RMSE and MAE (g) on unseen data
6 Data split & comparison?	80 % train / 20 % test (random_state = 42) · vs baseline



From L1

Task · Experience · Performance — now made precise.

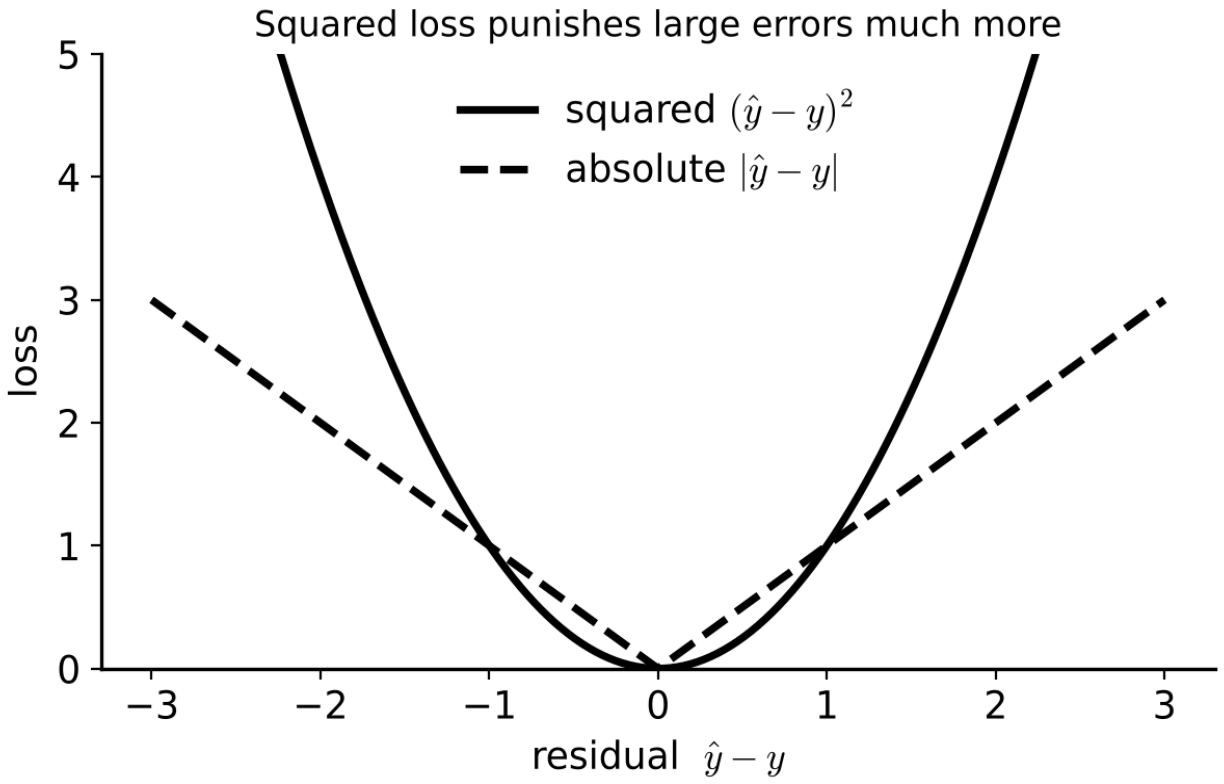


Write it first

Answer all six before coding — lab today, proposal in Week 8.

Loss functions: how bad is a prediction?

$$\ell_{\text{sq}} = (\hat{y} - y)^2 \qquad \ell_{\text{abs}} = |\hat{y} - y| \qquad \ell_{0\text{-}1} = \mathbf{1}[\hat{y} \neq y]$$



Squared

smooth · Gaussian noise · outlier-sensitive

Absolute

robust to outliers · kink at zero

0–1

accuracy · flat → use surrogate losses

The true risk: what we really care about

$$R(h) = \mathrm{E}_{(\mathbf{x},\, y)\,\sim\,\mathcal{D}}[\ell(h(\mathbf{x}),\ y)]$$

What it means

average loss over every example the world could produce — the generalisation error

The catch

$\mathcal{D}$ is unknown, so R(h) cannot be computed — only estimated

Population matters

a different population (new island, new hospital) has a different risk — distribution shift, Week 15

Empirical risk minimisation (ERM)

$$\hat{R}_n(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(\mathbf{x}_i),\ y_i)$$

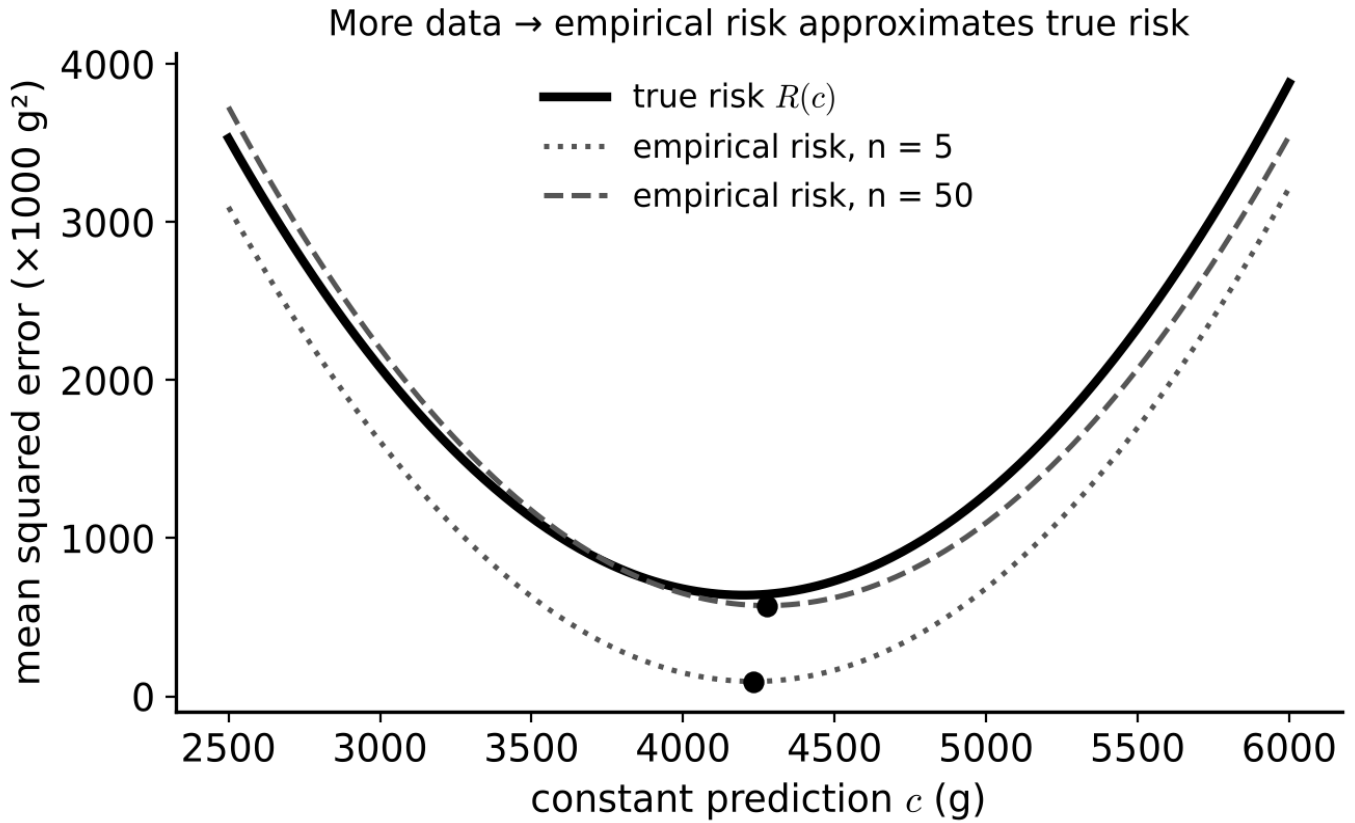
$$\hat{h} = \arg\min_{h \in \mathcal{H}}\ \hat{R}_n(h)$$

Law of large numbers

more data → $\hat{R}_n(h)$ closer to $R(h)$

Training risk is optimistic

→ always evaluate $\hat{h}$ on unseen data



Synthetic data with mean 4200 g, sd 800 g. Dots: minimum of each empirical curve.

What can go wrong with ERM?

$\mathcal{H}$ too small

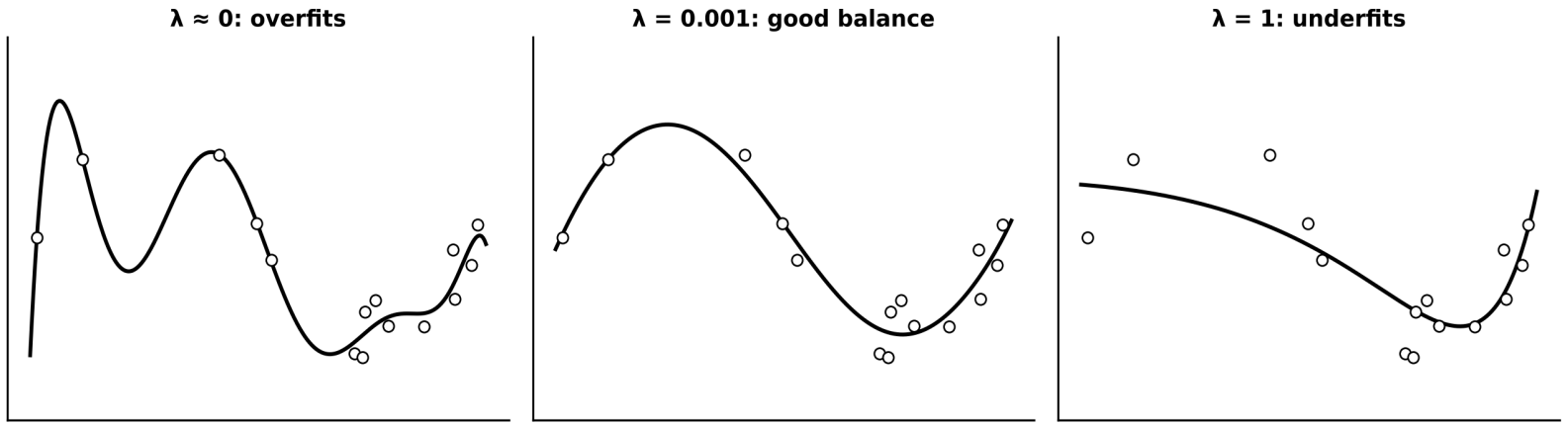
Approximation error

The best hypothesis in $\mathcal{H}$ is still poor. Underfitting: train and test error both high.

$\mathcal{H}$ too large for n

Estimation error

ERM fits noise in the sample. Overfitting: low train error, high test error.



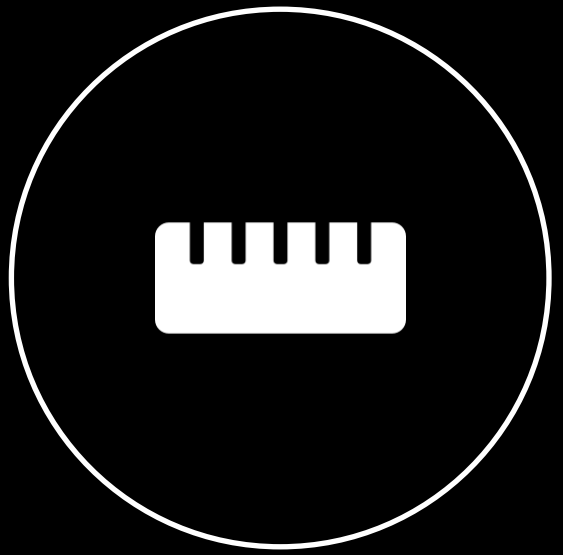
Coming up

Bias–variance in Week 4. Today: regularisation as a remedy (Part 06).

03

Baselines

The simplest ERM — and the bar every model must clear



Why every project needs a baseline

Regression

predict the training mean (or median)

```
DummyRegressor(strategy="mean")
```

Classification

predict the most frequent class

```
DummyClassifier(
  strategy="most_frequent")
```

Time series

predict the last observed value

```
y_hat[t] = y[t-1]
```

Is the model learning anything?

A model that can't clearly beat the baseline is useless — or has a bug (leakage, misaligned rows, missing intercept).

Course rule

Every lab, project and report states a baseline and compares against it.

ERM with constants: the mean minimises squared error

$$\frac{d}{dc}\frac{1}{n}\sum_i (y_i - c)^2 = -\frac{2}{n}\sum_i (y_i - c) = 0 \;\Rightarrow\; c^* = \bar{y}$$

$$\arg\min_c \; \frac{1}{n}\sum_i |y_i - c| = \text{median}(y_1, \ldots, y_n)$$

Squared loss

best constant = mean · pulled by outliers

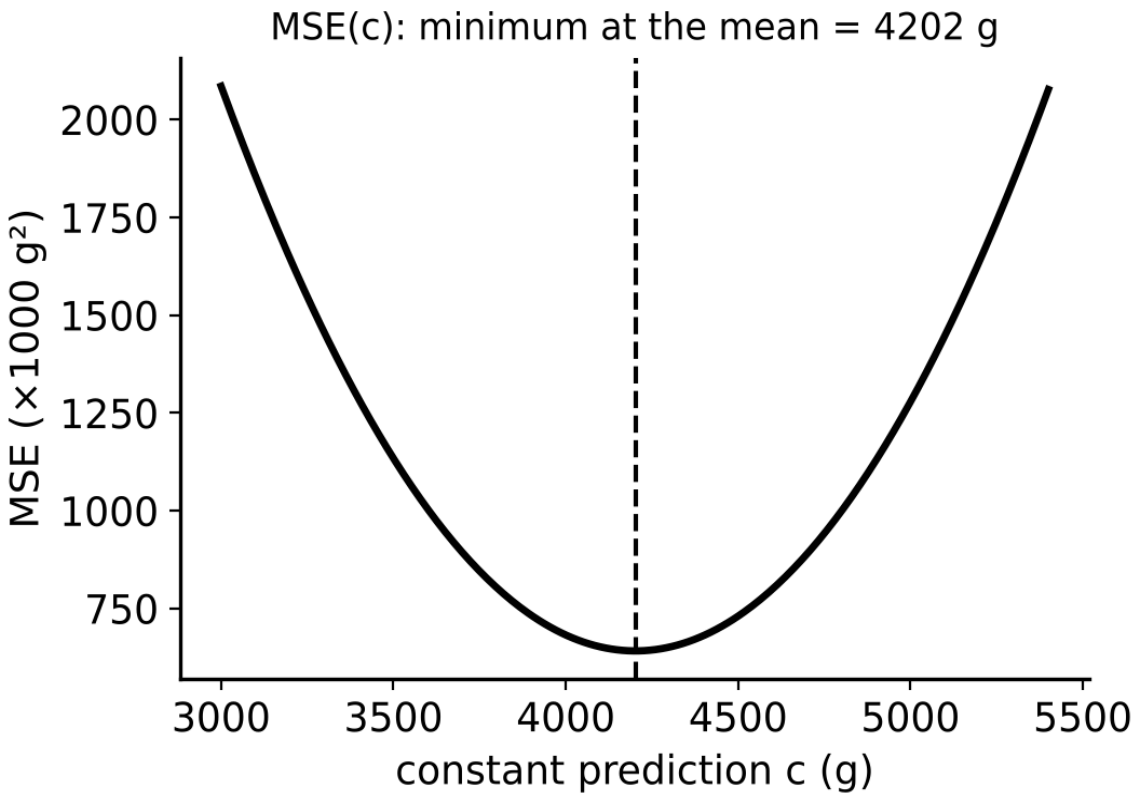
Absolute loss

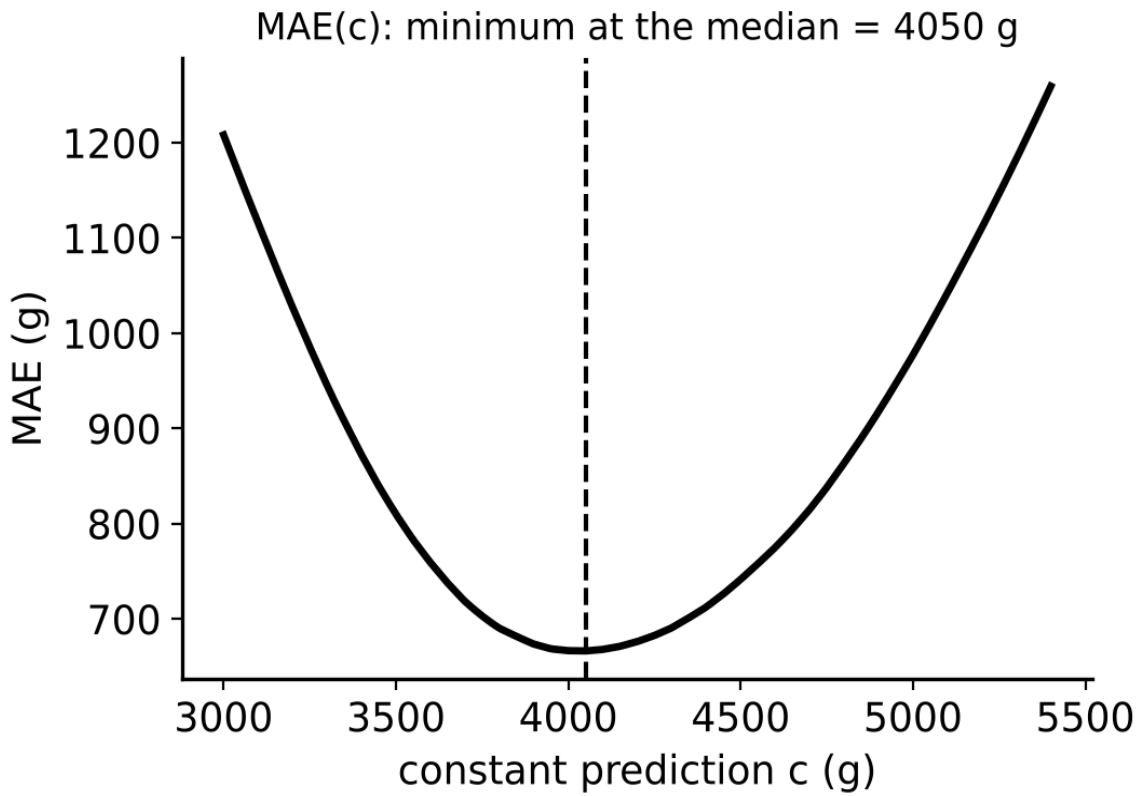
best constant = median · robust to outliers

Check

second derivative = 2 > 0 → minimum

The baseline on real data



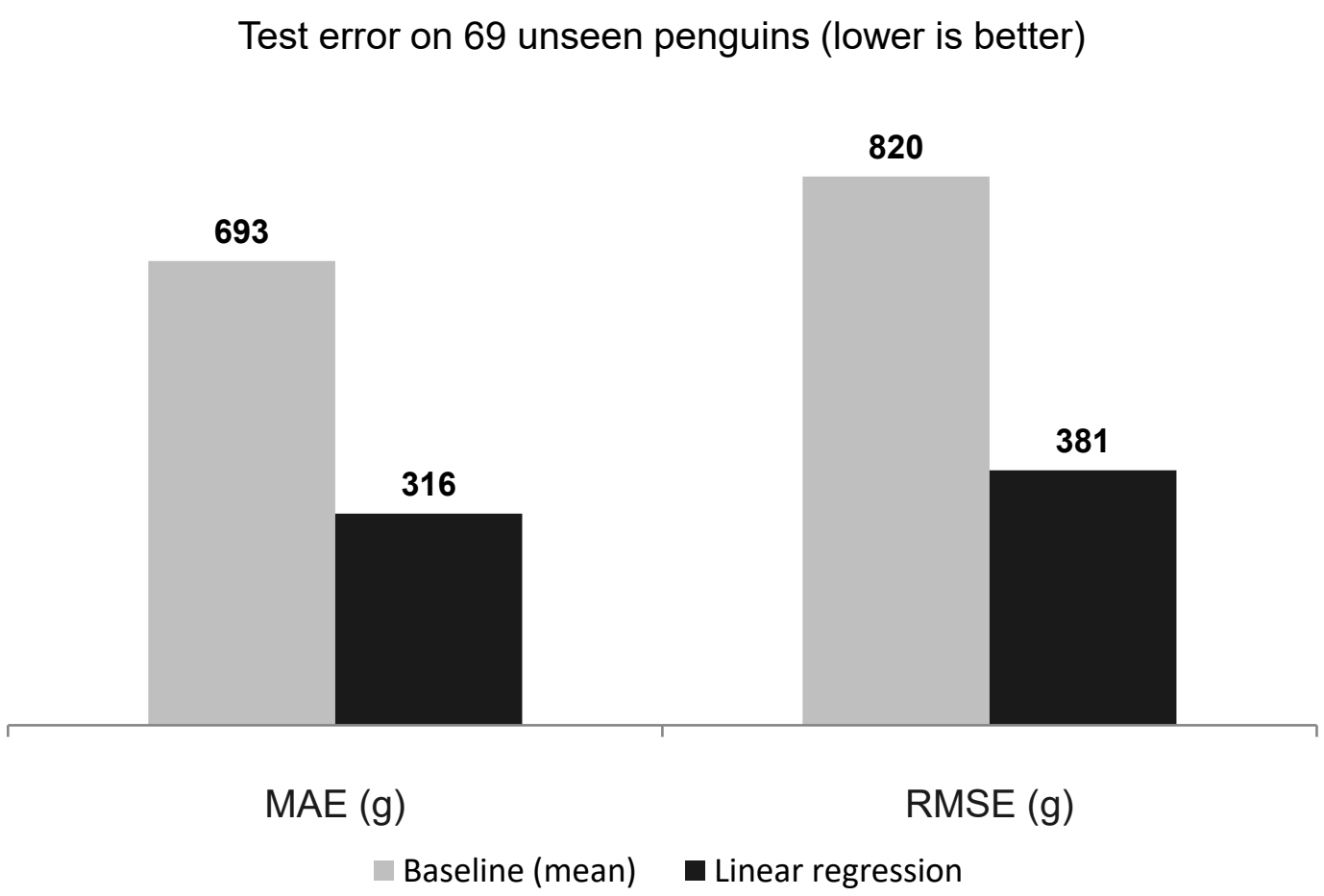


4202 g mean = best constant (MSE)

4050 g median = best constant (MAE)

801 g baseline RMSE = std of y

Baseline vs model on unseen data




```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression

base = DummyRegressor(strategy="mean")
base.fit(X_train, y_train)
lin = LinearRegression().fit(X_train, y_train)
```

≈ 53 % less error

RMSE 381 g vs 820 g. R^2 = 0.78: flipper length explains most of the variation in mass.

04

Linear regression & least squares

ERM with lines and squared loss — solved exactly



The linear model

$$\hat{y} = w\,x + b$$

*Penguins: $\hat{y}$ = w · *flipper* + b*

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d = \tilde{\mathbf{x}}^\top \mathbf{w}$$

Bias trick

Add a feature $x_0 = 1$ to every example. The intercept becomes an ordinary weight w_0.

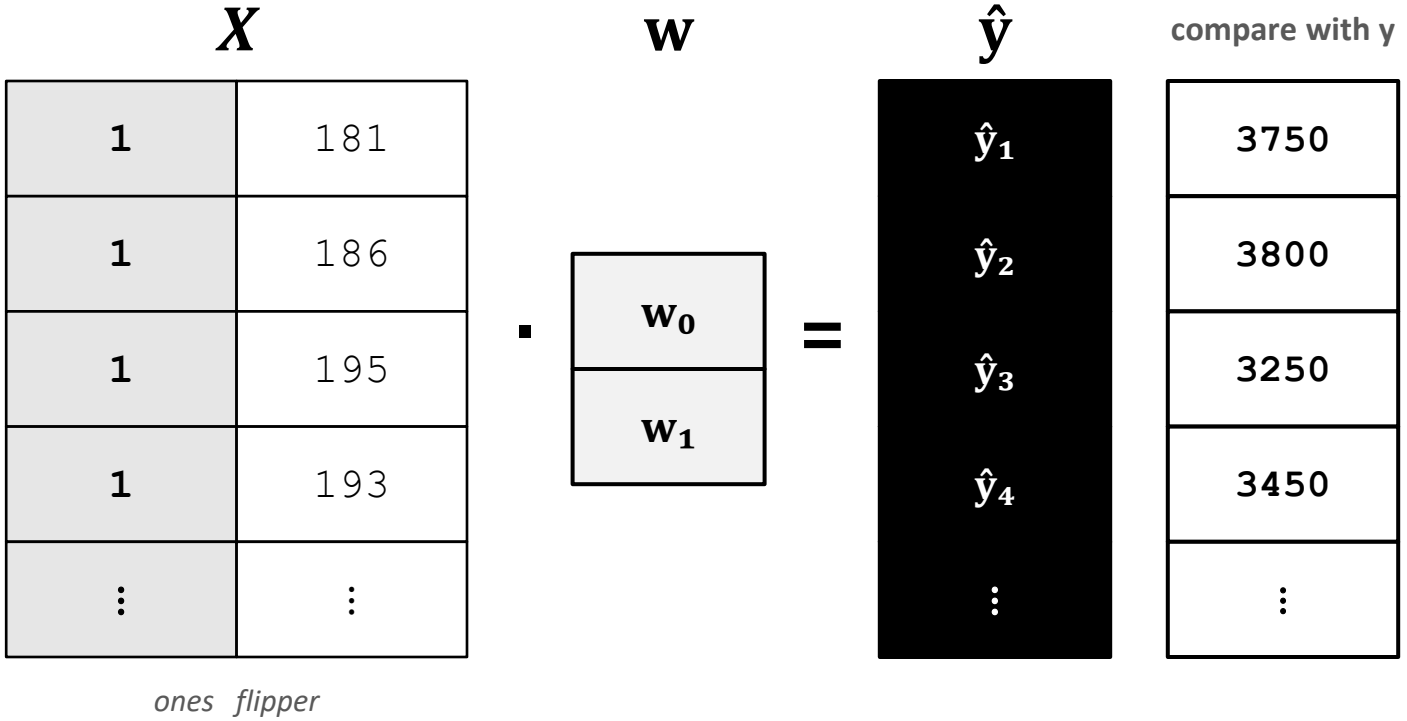
Interpretation

w_j = change in $\hat{y}$ when x_j rises by one unit, other features fixed. Units: target per feature unit.

Linear in parameters

Features such as x^2, log x or $x_1 x_2$ make curves — still linear regression.

The design matrix: all examples in one equation




```
x = penguins['flipper_length_mm'].to_numpy()
y = penguins['body_mass_g'].to_numpy()
X = np.c_[np.ones(len(x)), x]   # bias trick
X.shape                         # (342, 2)
y_hat = X @ w
```

Most common bug

Forgetting the ones column forces the line through the origin. (sklearn adds it for you: fit_intercept=True.)

The least-squares objective

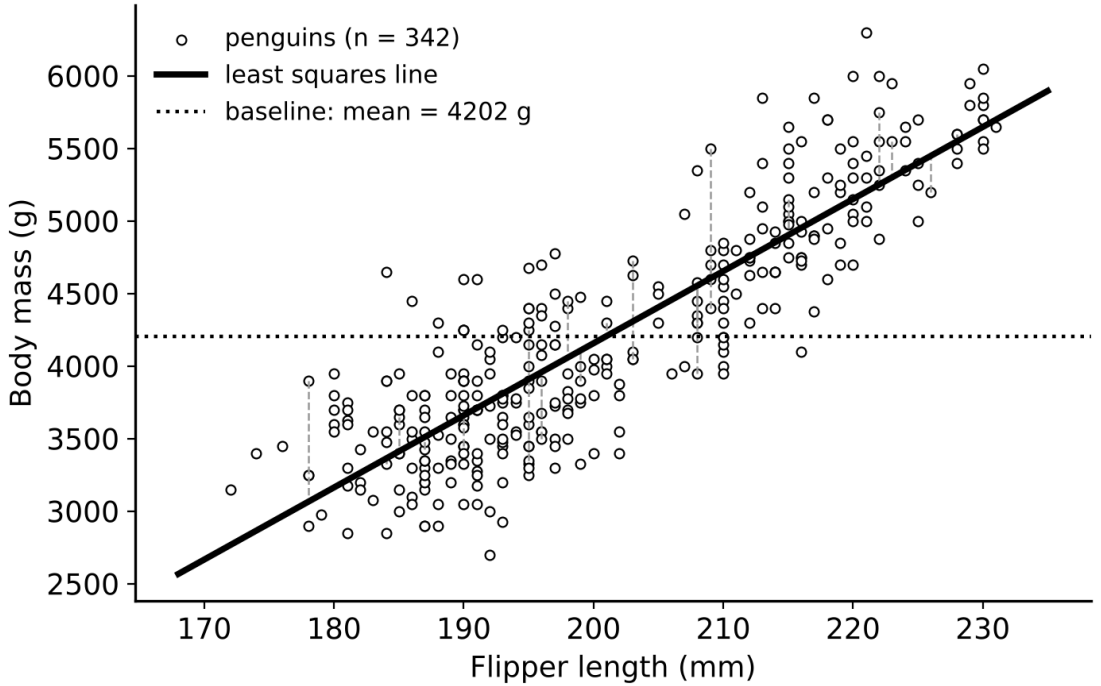
$$J(\mathbf{w}) = \frac{1}{n}\,\|X\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{n}\sum_{i=1}^{n}(\tilde{\mathbf{x}}_i^\top \mathbf{w} - y_i)^2$$

Vertical residuals

We minimise errors in the target, so distances are measured vertically — not perpendicular to the line.

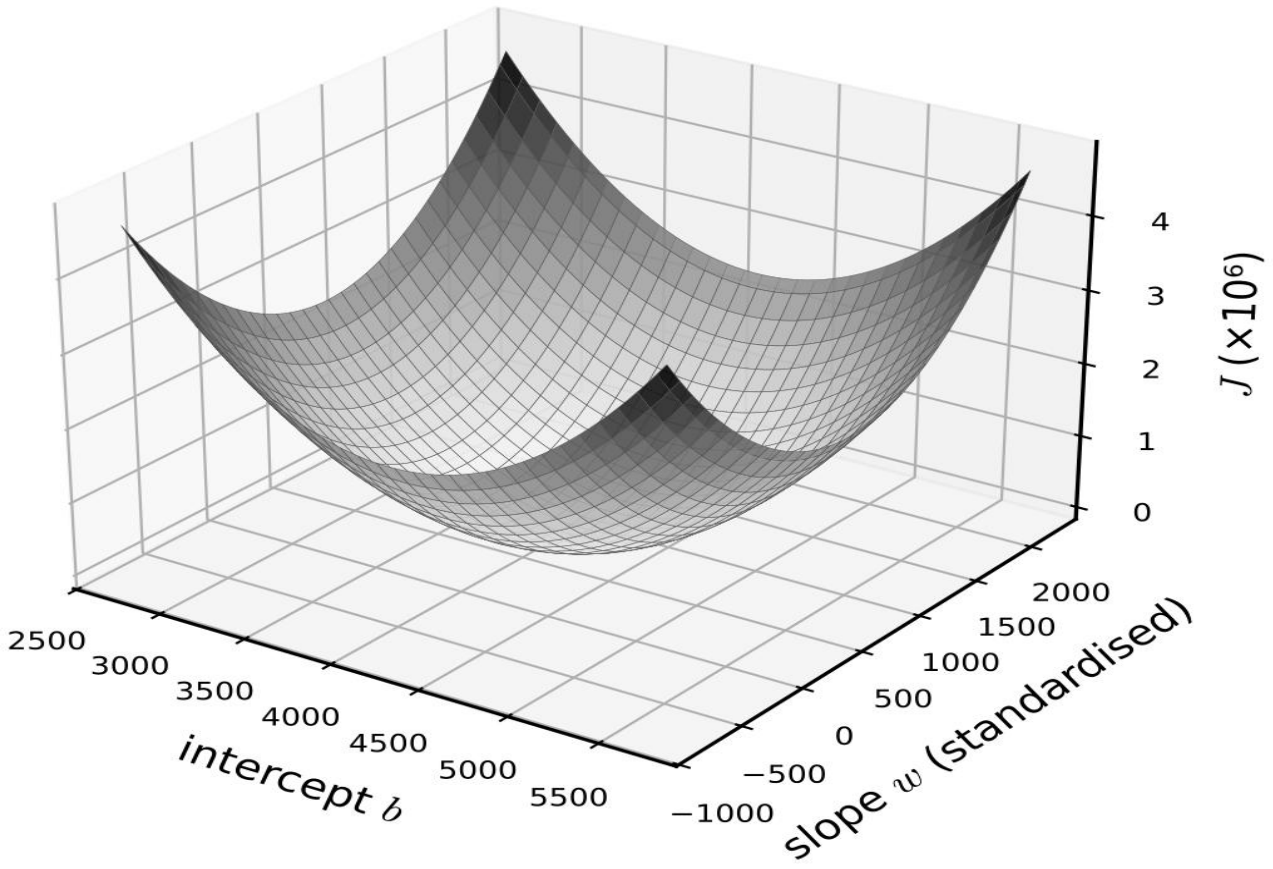
Over 200 years old

Legendre (1805) and Gauss (1809) fitted orbits of comets and planets with least squares.



342 penguins; dashed = residuals for 25 random penguins.

The loss surface is a convex bowl



Convex bowl

quadratic in (b, w) → exactly one minimum

Shape depends on scaling

standardised → round · raw mm → narrow valley

Two ways down

set the gradient to zero, or walk downhill

Deriving the normal equations

1

$$J(\mathbf{w}) = \frac{1}{n} \|X\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{n} \sum_{i=1}^{n} (\tilde{\mathbf{x}}_i^\top \mathbf{w} - y_i)^2$$

objective

2

$$\nabla J(\mathbf{w}) = \frac{2}{n} X^\top (X\mathbf{w} - \mathbf{y})$$

gradient (Part 01 rules)

3

$$X^\top X \hat{\mathbf{w}} = X^\top \mathbf{y} \quad \Rightarrow \quad \hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

set to zero → solve

One feature: slope $\hat{w} = \text{cov}(x, y) / \text{var}(x)$ intercept $\hat{b} = \bar{y} - \hat{w} \cdot \bar{x}$ (the line passes through the point of means)

Least squares on the penguins

$$\widehat{\text{mass}} \approx -5781 + 49.7 \times \text{flipper}$$

49.7

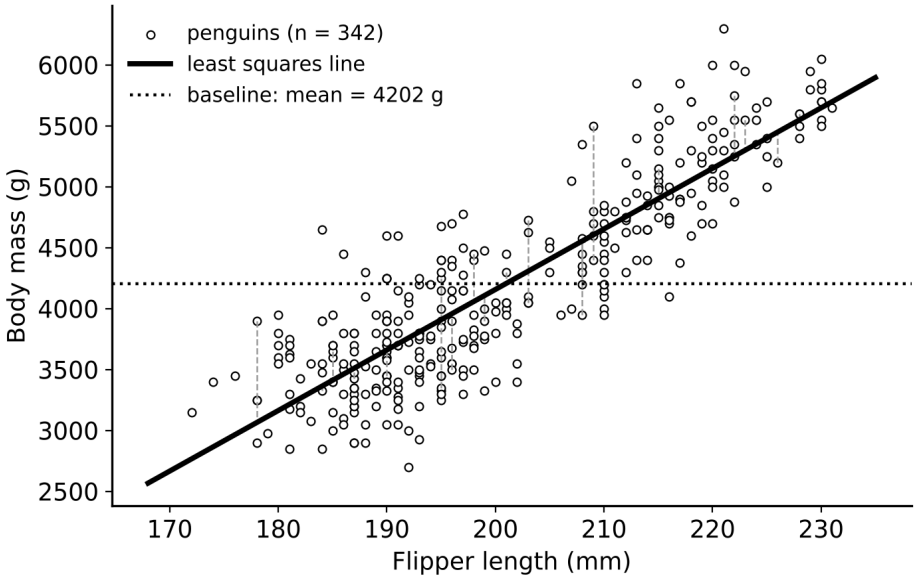
g per mm of flipper

4156 g

predicted at 200 mm

5150 g

predicted at 220 mm



Don't extrapolate

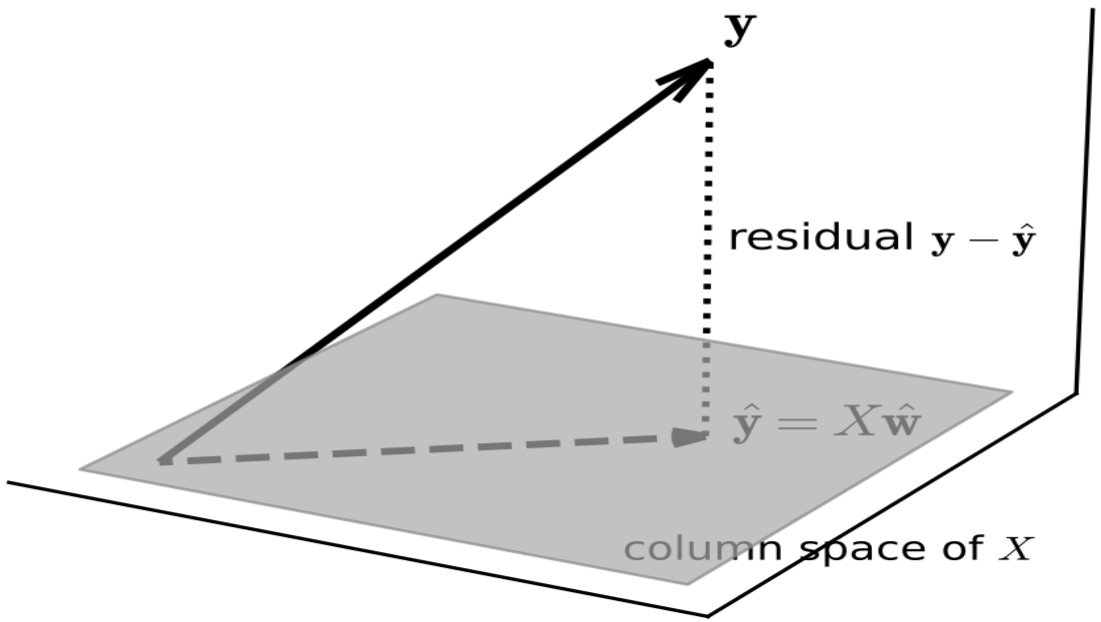
Data covers 172–231 mm. At 100 mm the line predicts negative mass.

Association ≠ causation

The slope describes co-variation, not cause and effect.

```
w = np.linalg.solve(X.T @ X, X.T @ y)
w                    # [-5780.83, 49.69]
w[0] + w[1] * 200    # 4156.3
```

Geometry: least squares is a projection



Column space

every Xw is a combination of X's columns — a flat subspace

Projection

ŷ = the closest point to y in that subspace

Normal equations = orthogonality

$X^T(y - \hat{y}) = 0 \rightarrow$ residuals sum to zero

Computing the solution: three ways, one answer

```
# 1. normal equations
w1 = np.linalg.solve(X.T @ X, X.T @ y)

# 2. least-squares solver
w2, *_ = np.linalg.lstsq(X, y, rcond=None)

# 3. scikit-learn (no ones column!)
lr = LinearRegression().fit(x.reshape(-1, 1), y)
lr.intercept_, lr.coef_

# all three: -5780.83, 49.69
```

Step	Cost
form X^TX	$\approx n \cdot d^2$
solve d×d system	$\approx d^3$
one GD step	$\approx n \cdot d$

→

Why not always closed form?

- $d = 10^6 \rightarrow d^3$ infeasible
- Lasso, logistic regression, neural networks: no closed form
- → gradient descent (Part 05)

Evaluating a regression model: R^2, MAE and RMSE

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Metric	Units	Penguins (train)	Penguins (test)
R^2	none	0.759	0.782
RMSE	grams	393	381
MAE	grams	313	316
Baseline RMSE	grams	801	820

$R^2 = 1$

perfect fit

$R^2 = 0$

no better than the mean

$R^2 < 0$

worse than the mean (possible on test data)

Always report: an error in target units + R^2 + the baseline.

Assumptions and pitfalls of linear regression



Non-linearity

residual plot shows a pattern → add features or a more flexible model



Outliers

squared loss lets a few points tilt the line



Extrapolation

outside the training range the line is a guess



Collinearity

correlated features → unstable, uninterpretable weights; ridge stabilises



Scales

compare weights only after standardising features



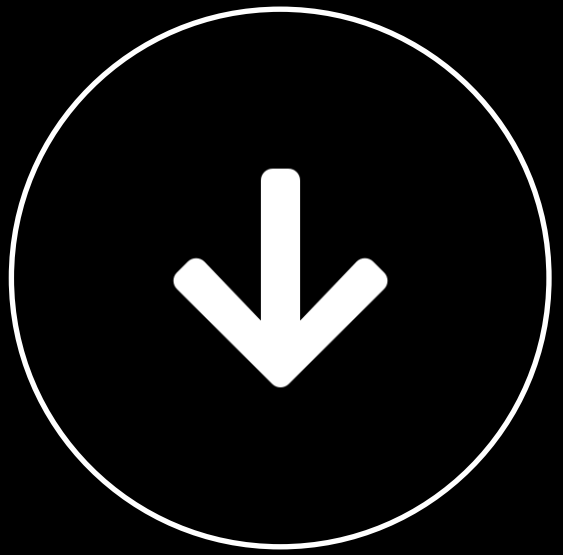
Causation

a slope is an association, not an effect

05

Gradient descent

Walking downhill when there is no formula — or the formula is too expensive



Why iterative optimisation?



Cost

d^3 is impossible for millions of features; a GD step is only ≈ n·d



No formula

lasso, logistic regression, neural networks have no closed-form solution



Big / streaming data

SGD learns from one example or a small batch at a time

From linear regression to transformers: every model in Weeks 3–14 is trained by gradient descent or a close relative (SGD with momentum, Adam). Linear regression is where we can check it against the exact answer.

The gradient descent algorithm

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$$

$$\nabla J(\mathbf{w}) = \frac{2}{n} X^{\mathsf{T}} (X\mathbf{w} - \mathbf{y})$$

1 **Predict**
$\hat{y} = Xw$

2 **Residuals**
$r = \hat{y} - y$

3 **Gradient**
$g = (2/n)\, X^{\mathsf{T}} r$

4 **Step**
$w \leftarrow w - \eta g$

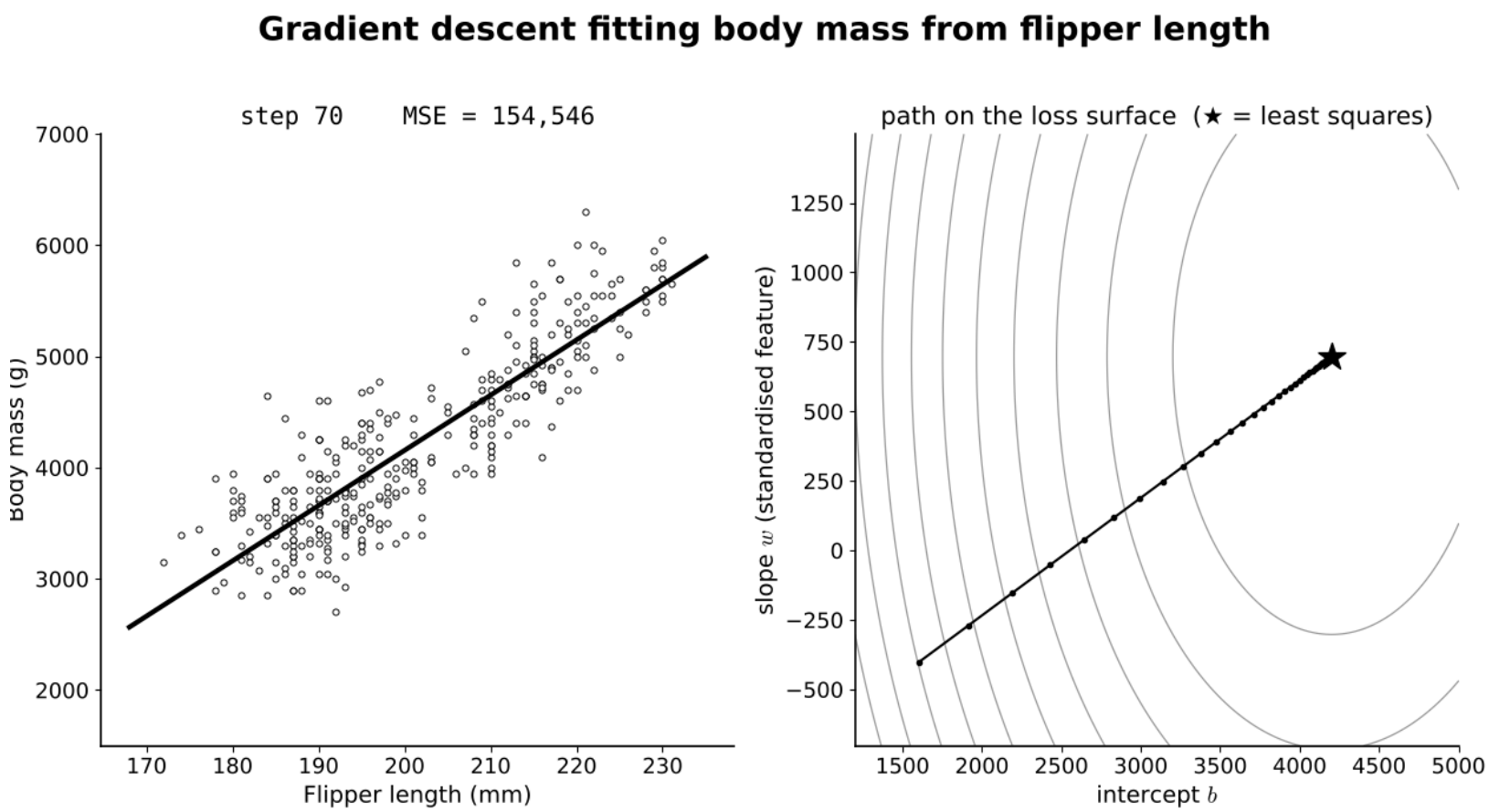
```
w = np.zeros(X.shape[1])
for step in range(n_steps):
    y_hat = X @ w            # 1
    r = y_hat - y            # 2
    g = 2 / n * X.T @ r      # 3
    w = w - eta * g          # 4
```



Two choices

the learning rate η and when to stop — next slides.

Video: gradient descent fitting the penguins

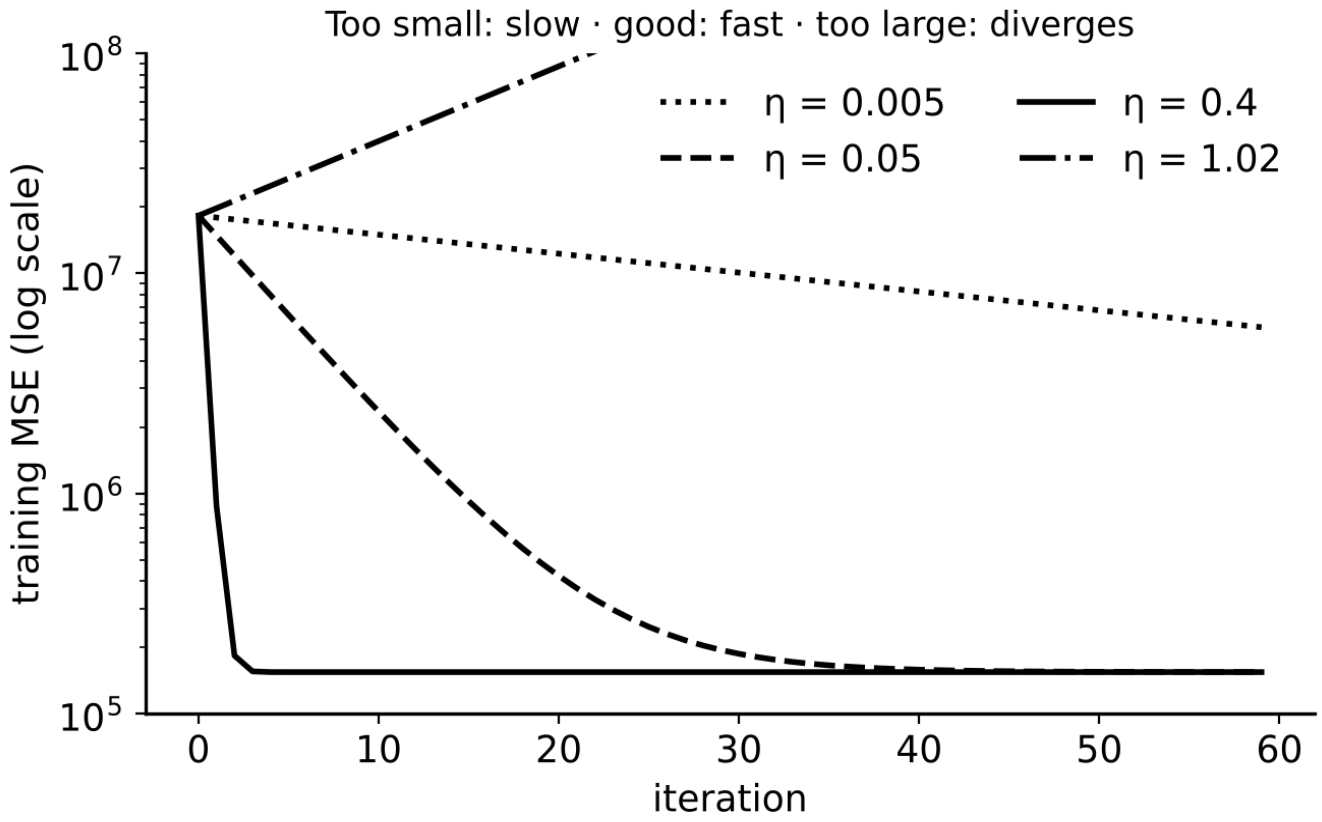


What to watch for

- The line rises and rotates towards the data
- MSE falls fast, then slowly
- Steps shrink as the gradient shrinks
- The path ends exactly at ★ (least squares)

Click to play (≈6 s). Animation generated for this lecture from the penguins data; η = 0.06.

The learning rate: the most important hyperparameter



Standardised penguin data; training MSE (log scale) for four learning rates.

Too small

slow — wastes computation

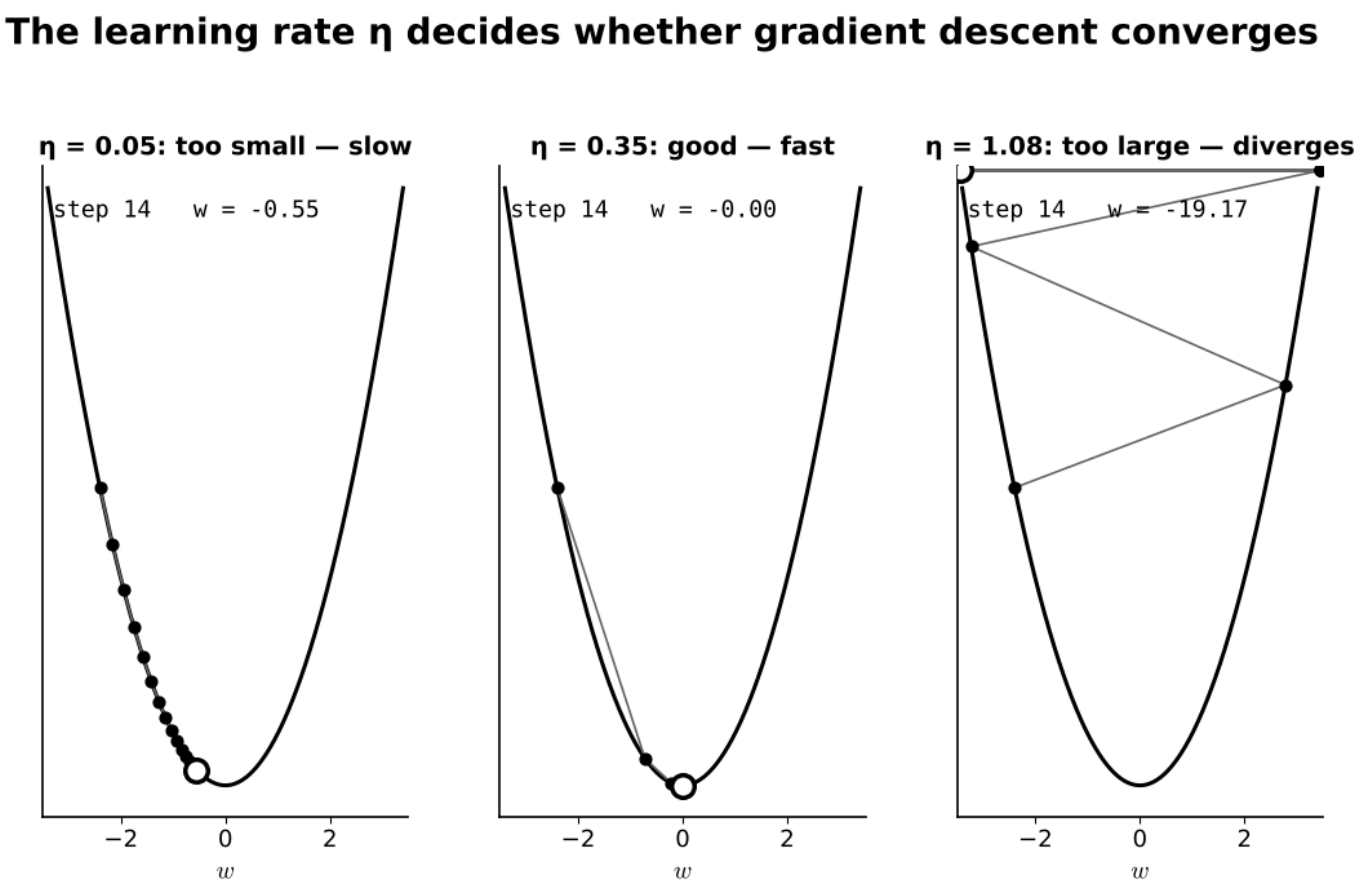
Too large

overshoots, diverges → loss = nan

Recipe

log grid 0.001 → 1; plot the loss curve; keep the largest η that decreases smoothly

Video: three learning rates on one bowl

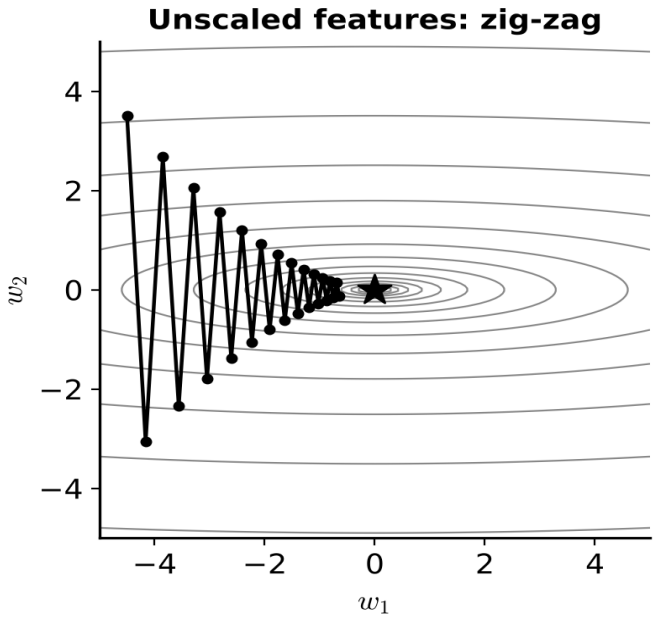


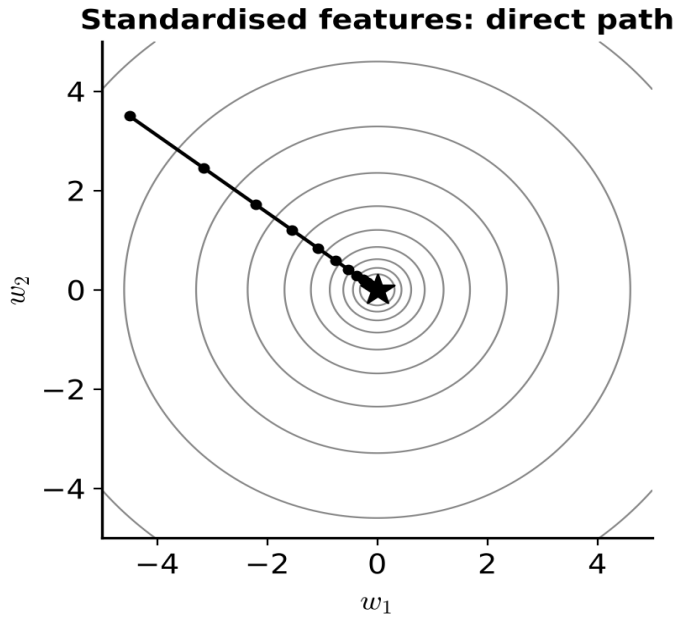
Click to play (≈5 s). Animation generated for this lecture.

The arithmetic

- Update: $w \leftarrow (1 - 2\eta)\, w$
- $|1 - 2\eta| < 1 \rightarrow$ converges
- $|1 - 2\eta| > 1 \rightarrow$ diverges
- Steeper bowl → smaller safe η

Feature scaling: round bowls are easy





$$x_j' = \frac{x_j - \mu_j}{\sigma_j}$$

Fit on training data only

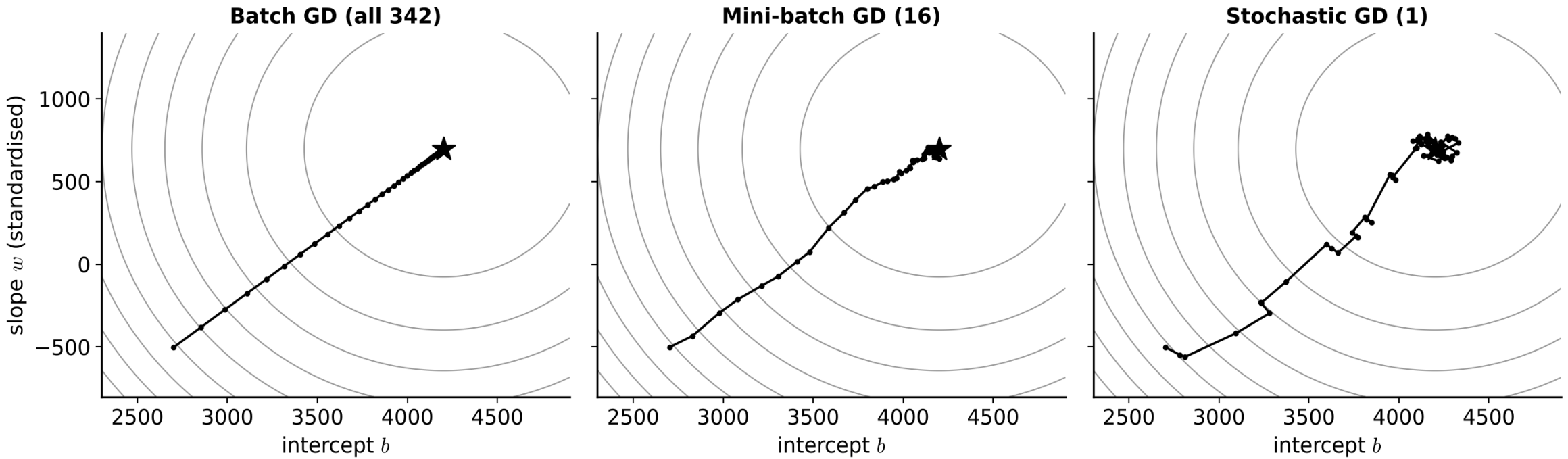
Use training μ and σ for the test set — otherwise leakage (Week 5).

```
mu, sigma = x_train.mean(), x_train.std()
x_train_s = (x_train - mu) / sigma
x_test_s  = (x_test  - mu) / sigma        # same mu, sigma!
```

Needed for

gradient descent · ridge · lasso · k-NN

Batch, mini-batch and stochastic gradient descent



Batch

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$$

Mini-batch

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\mathbf{w})$$

Stochastic (one random i)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i(\mathbf{w})$$

When to stop — and gradient descent vs the normal equations

	Normal equations	Gradient descent
Answer	exact, one shot	approximate, iterative
Hyperparameters	none	learning rate, stopping rule
Feature scaling	not needed	strongly recommended
Cost	$\approx n \cdot d^2 + d^3$	$\approx n \cdot d$ per step
Huge d / streaming	no	yes (SGD)
Other losses / models	linear regression only	almost any differentiable loss



Stopping rules

- Max iterations / epochs
- $\|\nabla J\| <$ tolerance
- Loss change < tolerance
- Early stopping on a validation set (also a regulariser — Week 4)

Gradient descent from scratch in NumPy

```
x_s = (x - x.mean()) / x.std()        # standardise
X = np.c_[np.ones(n), x_s]            # bias trick
w = np.zeros(2)
eta = 0.1
for step in range(200):
    grad = 2 / n * X.T @ (X @ w - y)
    w = w - eta * grad

w                                     # [4201.75, 697.64]
slope = w[1] / x.std()                # 49.69 g/mm
intercept = w[0] - slope * x.mean()   # -5780.8
```

4201.75

intercept (standardised) = mean mass

49.69

g per mm after converting back

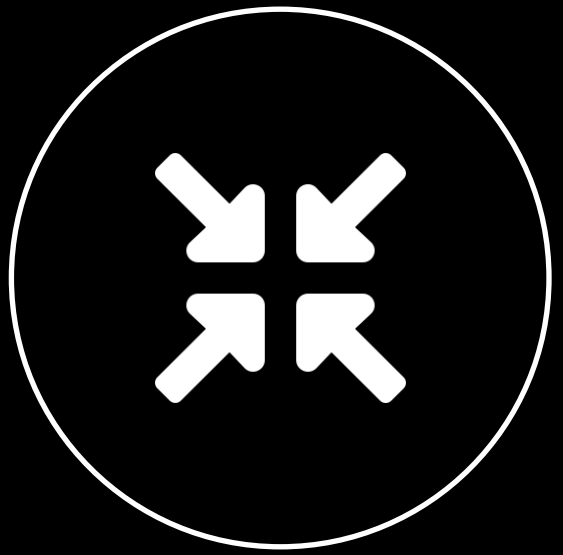
✓

matches the normal equations

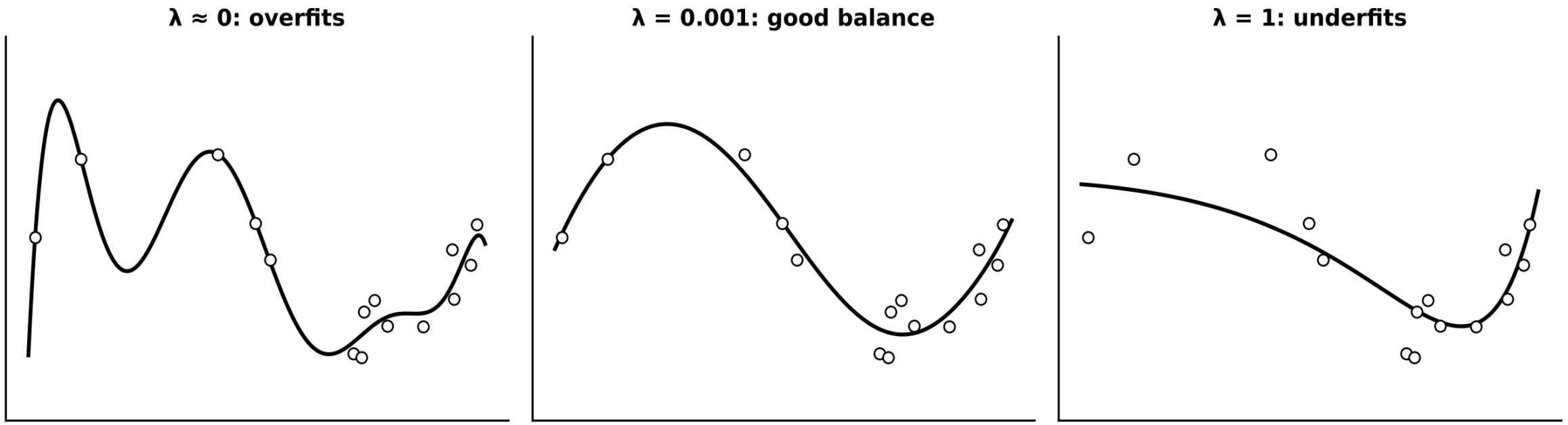
06

Regularisation: ridge & lasso

Trading a little training error for much better generalisation



Why regularise?



15 noisy points from a sine curve; degree-12 polynomial features (standardised) with ridge penalty λ.

minimise **training error + λ · penalty(w)**

Ridge regression (L2 penalty)

$$J_{\text{ridge}}(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

λ on the diagonal → always invertible for λ > 0

Don't penalise the intercept

shrink effects, not the average level

Standardise first

penalty must treat features equally

Choose λ on validation data

never on the test set (Week 5: CV)

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1.0)        # alpha = λ
ridge.fit(X_train_s, y_train)
ridge.coef_

# from scratch (no intercept penalty):
A = Xs.T @ Xs + lam * np.eye(d)
w = np.linalg.solve(A, Xs.T @ yc)
```

Xs: standardised features · yc: centred target (so no intercept is needed).

Ridge in gradient descent: weight decay

$$\mathbf{w} \leftarrow (1 - 2\eta\lambda)\,\mathbf{w} - 2\eta\, X^{\mathsf{T}}(X\mathbf{w} - \mathbf{y})$$



$(1 - 2\eta\lambda)\mathbf{w}$

decay: every weight shrinks a little towards 0 at every step



Data step

$-\,2\eta\, X^{\mathsf{T}}(Xw - y)$ pushes the weights towards a better fit



Balance

weights survive only if the data keeps pushing them — weight decay in deep learning (Week 11)

Lasso regression (L1 penalty)

$$J_{\text{lasso}}(\mathbf{w}) = \frac{1}{2n}\|X\mathbf{w} - \mathbf{y}\|_2^2 + \alpha\,\|\mathbf{w}\|_1$$

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
(lasso.coef_ != 0).sum()   # features kept
```

Sparsity

As α grows, more weights become exactly 0 — the model selects its own features.

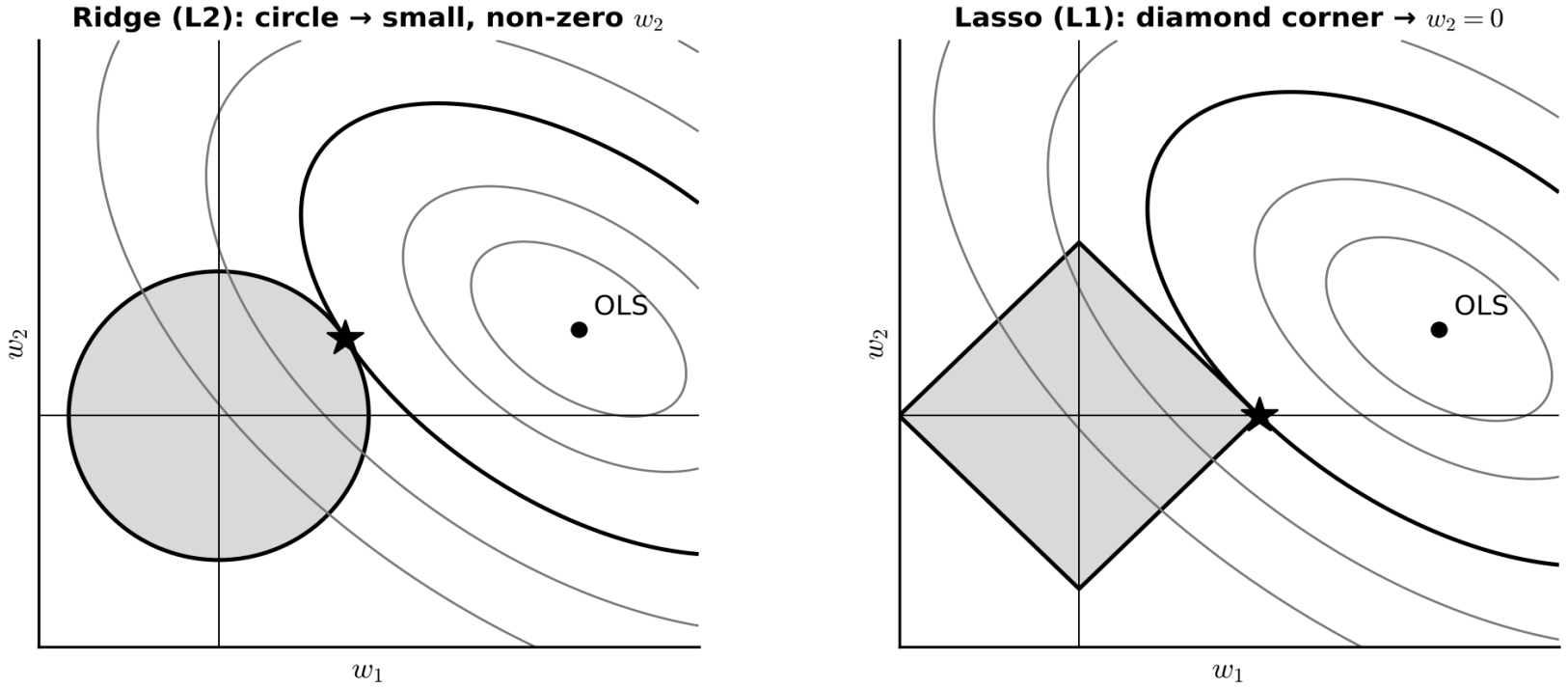
No closed form

|w| has a kink at 0 → solved by coordinate descent, one weight at a time.

History

Introduced by Robert Tibshirani (1996): least absolute shrinkage and selection operator.

Why lasso gives zeros: the geometry



Ridge (disc)

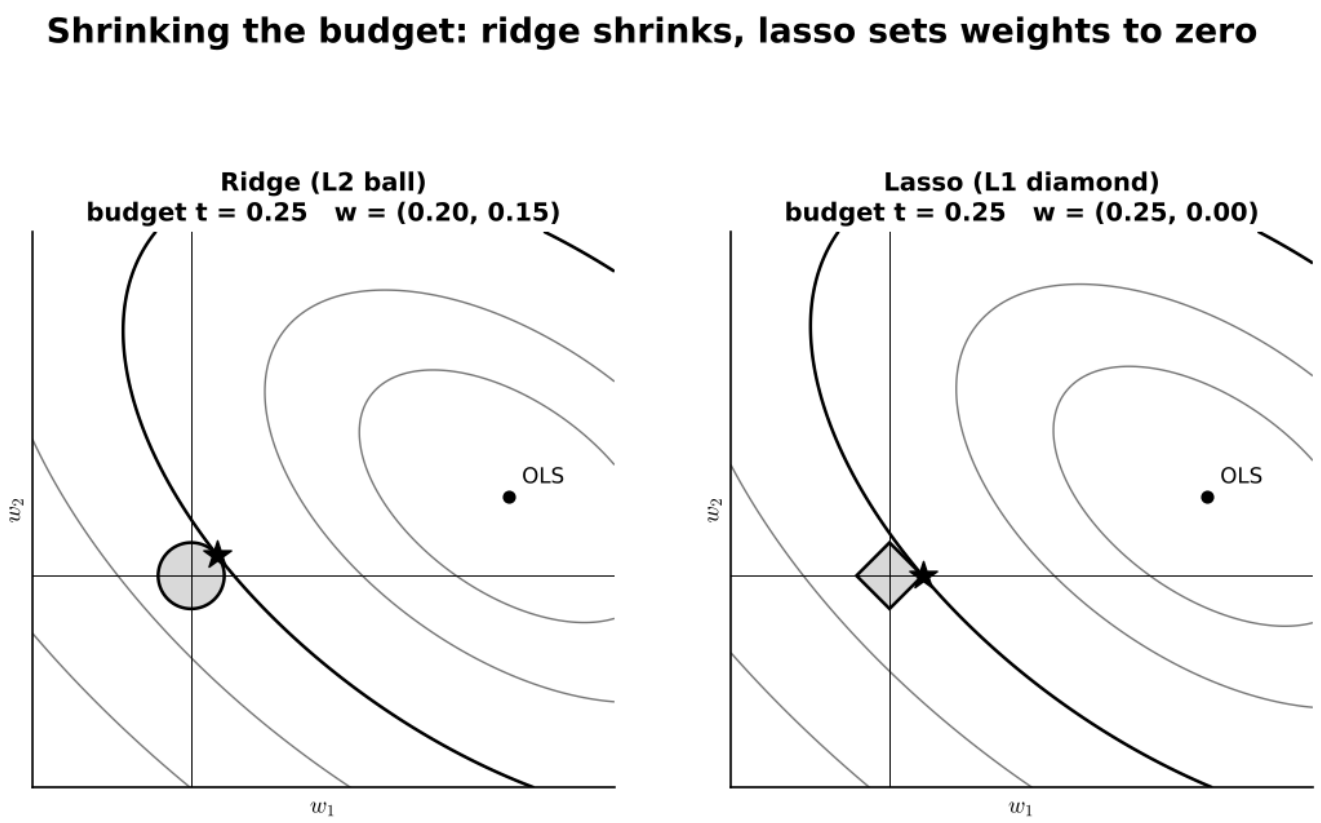
smooth boundary → w = (0.84, 0.54): both non-zero

Lasso (diamond)

corner on the axis → w = (1.2, 0): exactly zero

Many dimensions → many corners and edges with zeros → sparse solutions.

Video: shrinking the budget

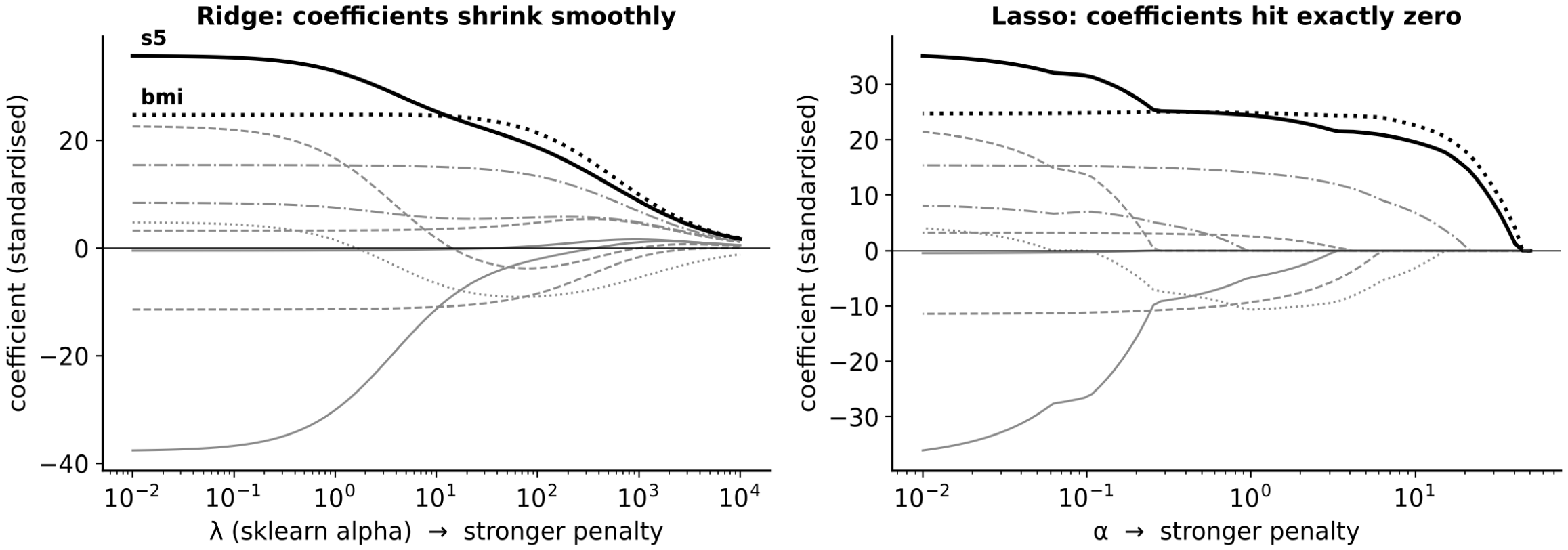


What to watch for

- Large budget: OLS solution
- Ridge slides smoothly towards 0
- Lasso: w_2 snaps to 0.00 and stays
- End: one feature eliminated by lasso

Click to play (≈6 s). Animation generated for this lecture.

Coefficient paths on real data

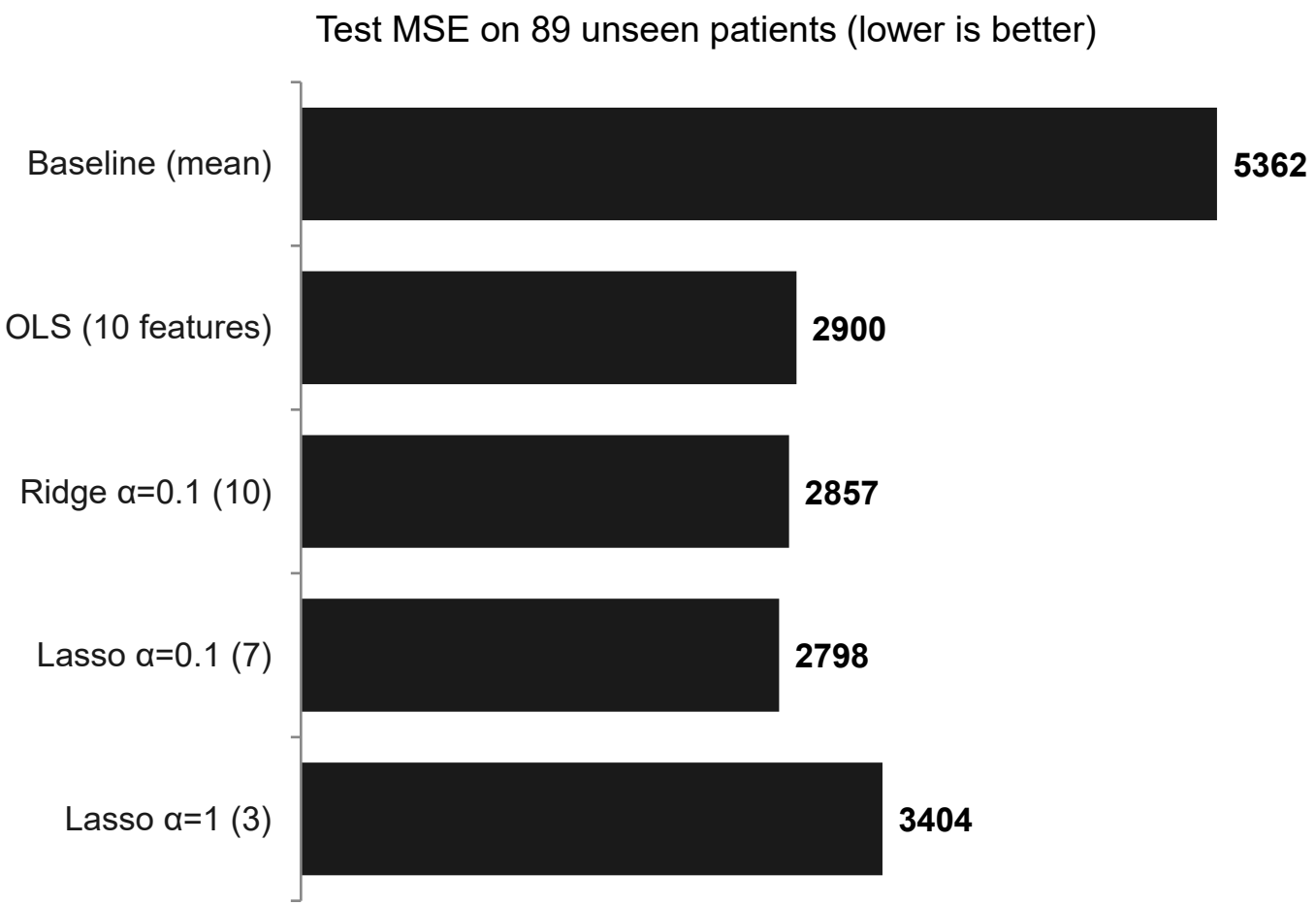


scikit-learn diabetes dataset (Efron et al., 2004), features standardised. Bold: bmi and s5.

Ridge: smooth shrinkage, never exactly zero

Lasso: zeros one by one; bmi and s5 survive

Ridge and lasso on the diabetes test set



Model	Test R^2	Features
OLS	0.453	10
Ridge α = 0.1	0.461	10
Lasso α = 0.1	**0.472**	**7**
Lasso α = 1	0.358	3
Lasso α = 2	0.054	1

Caution

Small differences, one split of 89 patients → use cross-validation (Week 5).

Ridge, lasso or elastic net?

	Ridge (L2)	Lasso (L1)	Elastic net
Penalty	$\lambda\|w\|_2^2$	$\alpha\|w\|_1$	mix of both
Exact zeros?	no	yes	yes
Feature selection	no	yes	yes, grouped
Closed form	yes	no	no
Correlated features	shares weight	picks one (unstable)	more stable
scikit-learn	Ridge(alpha)	Lasso(alpha)	ElasticNet(alpha, l1_ratio)

Elastic net:

$$\frac{1}{2n}\|X\mathbf{w}-\mathbf{y}\|_2^2+\alpha\rho\,\|\mathbf{w}\|_1+\frac{\alpha(1-\rho)}{2}\|\mathbf{w}\|_2^2$$

Choosing λ: a practical recipe

1

Split

train / test — lock the test set away

2

Validate

validation set or cross-validation

3

Scale

fit scaler on training part only

4

Grid

$\lambda = 10^{-3}, 10^{-2}, \ldots, 10^{3}$

5

Select

lowest validation error → refit

6

Report

test error, once

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV

model = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 13)))
model.fit(X_train, y_train)
model[-1].alpha_                # chosen λ
```



Never break

Never choose λ — or anything — by looking at the test set.

07

Lab & next steps

Implement it yourself — then compare with scikit-learn



Today's lab: from NumPy to regularised regression

A

NumPy

shapes, broadcasting, @ vs *

B

Baseline

mean & median, DummyRegressor

C

Normal equations

from scratch vs lstsq & sklearn

D

Gradient descent

loss curves, η, scaling, SGD

E

Ridge & lasso

closed form, paths, diabetes

F

Reflection

limitations, what you learned

Rule for every from-scratch result

Check it against a trusted library (np.linalg.lstsq, scikit-learn) and plot the loss curve.

Common pitfalls when implementing from scratch

Bug	Symptom	Fix
X * w instead of X @ w	shape (n, d), no error	use @; print shapes
missing column of ones	line forced through the origin	np.c_[np.ones(n), x]
η too large	loss grows, then nan	smaller η; plot the loss
unscaled features	zig-zag, very slow	standardise (train μ, σ)
scaler fitted on all data	optimistic test error (leakage)	fit on training data only
penalised intercept	predictions pulled towards 0	centre y / don't penalise w_0
y shape (n, 1) vs (n,)	residuals become n × n	y = y.ravel(); check shapes
gradient without 1/n	η behaves differently per dataset	use the mean, not the sum

Defence: print shapes · plot the loss · compare with a library · test on a tiny hand-computed example.

Watch and read before next week

WATCH



StatQuest — Gradient Descent, Step-by-Step



StatQuest — Regularization Part 1: Ridge (L2)



StatQuest — Regularization Part 2: Lasso (L1)



3Blue1Brown — Essence of Linear Algebra



READ

James et al. (2023) — ISLP
Ch. 3 Linear Regression · §6.2 Shrinkage Methods

Géron (2022) — Hands-On ML
Ch. 4 Training Models

Deisenroth, Faisal & Ong (2020)
Mathematics for Machine Learning — Ch. 2, 5, 9 · free at mml-book.github.io

Before Week 3
revise derivatives of exp and log

Key takeaways

1. Learning = ERM. Empirical ≠ true risk → always test on unseen data.
2. Always compute a baseline. The mean minimises squared error; R^2 compares with it.
3. Least squares solves $X^TXw = X^Ty$ — geometrically, a projection.
4. Gradient descent steps against ∇J; η and scaling decide convergence.
5. Ridge (L2) shrinks weights; lasso (L1) sets them exactly to zero.
6. Choose λ on validation data — never on the test set.

Choose $\mathcal{H}$ and ℓ, minimise the average training loss, control complexity with regularisation.

Exit ticket (3 minutes)

1. In one sentence: the difference between true risk and empirical risk, and why it matters.

2. Write the gradient-descent update for linear regression (MSE). What happens if η is too large?

3. 200 candidate features, only a handful relevant: ridge or lasso? Why?



Not graded

Write on paper or post in the course MS Teams channel. It helps me plan next week.

Thank you

Questions?



Next: Lab — NumPy, baseline, regression and gradient descent from scratch, ridge & lasso

Week 3: Logistic regression · maximum likelihood · multiclass classification · Naive Bayes

Contact: Behnam Kiani · Kiani-bme@hotmail.com · office hours by appointment